

anonymous

You are an expert at coding, linear algebra, machine learning, and deep learning. Your job is to solve the coding problem given to you by the user. Await for the user's response.



Llama

I'm ready to help. Please go ahead and provide the coding problem you'd like me to solve. What programming language and domain (e.g., linear algebra, machine learning, deep learning) is it related to? Let's get started!



anonymous

Image Blurring

Image blurring also called image smoothing, usually refers to making an image fuzzy. This filtering is typically used to remove noise in the image. There are various types of image blurring filters, but the three most common are Averaging, Gaussian blurring, and Median filtering.

We will implement Averaging filtering in this project. Averaging filtering is also called moving averaging in 1-D. This filter works by placing a mask over an image and then taking the average of all the image pixels covered by the mask and replacing the central pixel with that value.

If the kernel size of the image filter is $n \times n$, then the size of each element in the kernel matrix is $\frac{1}{n^2}$. Also, the sum of all the elements in the kernel matrix will be 1. So, if the kernel size is 3×3 , kernel will be as follows.

```
$
\frac{1}{9} \times \left[
\begin{array}{ccc}
1 & 1 & 1 \\
1 & 1 & 1 \\
1 & 1 & 1
\end{array}
\right]
$
```

python

Copy code

```
def averaging_filtering(image, filter_size=3):
#####
```

```

# TODO: Implement the averaging filter with the given filter size.
# Hint: You can use np.ones
#####;
#####;
#                                     END OF YOUR CODE
#####;
output = ndimage.convolve(image, kernel)
return output

```

Llama

Implementing Averaging Filtering for Image Blurring

To implement the averaging filtering, we need to create a kernel matrix based on the given filter size. The kernel matrix is a square matrix with all elements being 1, and its size is determined by the `filter_size` parameter. We then normalize this matrix by dividing all its elements by the square of the `filter_size`.

Here's how you can implement the `averaging_filtering` function:

python

[Copy code](#)

```

import numpy as np
from scipy import ndimage

def averaging_filtering(image, filter_size=3):
    """
    Applies an averaging filter to the given image.

    Args:
    - image: The input image to be filtered.
    - filter_size: The size of the averaging filter. Defaults to 3.

    Returns:
    - The filtered image after applying the averaging filter.
    """
    #####;
    # TODO: Implement the averaging filter with the given filter size.
    # Hint: You can use np.ones
    #####;
    # Create a kernel matrix filled with ones, with the size of filter_size
    kernel = np.ones((filter_size, filter_size))

    # Normalize the kernel by dividing all elements by the square of filter.
    kernel = kernel / (filter_size ** 2)

```

```
#####
#####
#                                     END OF YOUR CODE
#####
# Convolve the image with the kernel using ndimage.convolve
output = ndimage.convolve(image, kernel, mode='nearest')
return output
```

Explanation

1. **Creating the Kernel Matrix:** We use `np.ones((filter_size, filter_size))` to create a 2D array filled with ones, with dimensions `filter_size` by `filter_size`.
2. **Normalizing the Kernel:** To ensure that the sum of all elements in the kernel is 1, we divide the kernel by `(filter_size ** 2)`. This is because the sum of all elements in the kernel before normalization is `filter_size * filter_size`.
3. **Applying the Filter:** We use `ndimage.convolve` from the SciPy library to convolve the input image with the kernel. The `mode='nearest'` parameter is used to specify how the boundaries are handled.

Example Usage

python

[Copy code](#)

```
import matplotlib.pyplot as plt
from scipy import misc

# Load an example image
image = misc.ascent()

# Apply the averaging filter with a filter size of 5
filtered_image = averaging_filtering(image, filter_size=5)

# Display the original and filtered images
fig, axes = plt.subplots(1, 2, figsize=(10, 5))
axes[0].imshow(image, cmap='gray')
axes[0].set_title('Original Image')
axes[1].imshow(filtered_image, cmap='gray')
axes[1].set_title('Filtered Image')
plt.show()
```

anonymous

Edge Detection

Next, we will implement a simple edge detection filter. Edge detection is an algorithm

that detects edges in an image. An edge in an image is a place where the brightness of the image changes abruptly or discontinuously. Several edge detection algorithms exist, such as the Canny edge detector, the Sobel filter and the Laplacian derivatives filter.

Here, we will use the 3×3 Laplacian kernel. This kernel, when convolved with the image, computes the 3×3 discrete Laplacian of the image:

```
$
\left[
\begin{array}{ccc}
0 & 1 & 0 \\
1 & -4 & 1 \\
0 & 1 & 0
\end{array}
\right]
$
```

python

[Copy code](#)

```
def edge_detecting(image):
    #####
    # TODO: Implement the Laplacian derivative filter.
    #####
    #####
    #
    #                                     END OF YOUR CODE
    #####
    output = ndimage.convolve(image, kernel)
    return output

edge_images, edge_titles = [gray_img], ['original']
edge_image = edge_detecting(gray_img)
edge_images.append(edge_image)
edge_titles.append(f'Edge Detection')

show_multiple_images(edge_images, edge_titles)
```

Llama

Implementing Edge Detection using Laplacian Derivative Filter

To implement the edge detection using the Laplacian derivative filter, we need to define the Laplacian kernel and then convolve it with the input image.

Here's how you can implement the `edge_detecting` function:

Here's how you can implement the `edge_detecting` function:

python

[Copy code](#)

```
import numpy as np
from scipy import ndimage

def edge_detecting(image):
    """
    Applies the Laplacian derivative filter to detect edges in the given image.

    Args:
    - image: The input image to detect edges in.

    Returns:
    - The image with edges detected after applying the Laplacian derivative filter.
    """
    #####
    # TODO: Implement the Laplacian derivative filter.
    #####
    # Define the Laplacian kernel
    kernel = np.array([
        [0, 1, 0],
        [1, -4, 1],
        [0, 1, 0]
    ])
    #####
    #                                     END OF YOUR CODE
    #####
    # Convolve the image with the kernel using ndimage.convolve
    output = ndimage.convolve(image, kernel, mode='nearest')
    return output

# Example usage:
edge_images, edge_titles = [gray_img], ['original']
edge_image = edge_detecting(gray_img)
edge_images.append(np.abs(edge_image)) # Take the absolute value to visualize edges
edge_titles.append(f'Edge Detection')

show_multiple_images(edge_images, edge_titles)
```

Explanation

1. **Defining the Laplacian Kernel:** We define the Laplacian kernel as a 3×3 numpy array according to the given specification.
2. **Applying the Filter:** We use `ndimage.convolve` from the SciPy library to convolve

the input image with the Laplacian kernel. The `mode='nearest'` parameter is used to specify how the boundaries are handled.

3. **Visualizing Edges:** Since the output of the Laplacian filter can have negative values, we take the absolute value of the output before visualizing it to properly represent the edges.

Notes

- The Laplacian filter is sensitive to noise, so it might be beneficial to apply some smoothing (like Gaussian blurring) to the image before edge detection.
- The example usage takes the absolute value of the edge detection output to visualize the edges. This is because the Laplacian filter output can have both positive and negative values, and we're interested in the magnitude of the edges.

The LLM gave fully correct solutions for this question.

anonymous

You are an expert at coding, linear algebra, machine learning, and deep learning. Your job is to solve the coding problem given to you by the user. Await for the user's response.



Llama

I'm ready to help. Please go ahead and provide the coding problem you'd like me to solve. I'll do my best to provide a well-structured and efficient solution, leveraging my expertise in coding, linear algebra, machine learning, and deep learning. I'm waiting for your response!



anonymous

Here is some context. There are no problems to solve here.

HW3 - Q6: Exploring Inductive Bias of Convolutional Neural Networks and Systematic Experimentation in Machine Learning

In this homework, we will study 1) what is inductive bias and how it affects the learning process, and 2) how to conduct systematic experiments in machine learning. We will compare convolutional neural networks (CNNs) and multi-layer perceptrons (MLPs) extensively as an example to study these two topics.

I initially did not give the LLM this much context, but it performed poorly so I gave it this context.

1. Inductive Bias

What is inductive bias? It is the assumption that the learning algorithm makes about the problem domain. Suppose that we build a machine learning system. We want to leverage the specific knowledge about the problem domain to make the learning process **more efficient** and the system **generalize much better** with fewer parameters. Let's be more precise. What do exactly **more efficient** and **generalize much better** mean? The learning process is more efficient 1) if we can learn the model with fewer parameters, 2) if we can learn the model with fewer data, and 3) if we can learn the model with fewer iterations. And the system generalizes much better if the model can generalize to the unseen data well.

We have already observed the power of inductive bias. We know that CNN generalizes better than MLP even with the same number of parameters. We partially concluded that is because CNN has the inductive bias that the model is translation invariant. We will study the inductive bias of CNN in more detail in this homework.

In this homework, we will use the edge detection task as an example to study the

In this homework, we will use the edge detection task as an example to study the inductive bias of CNN. We will compare CNN and MLP extensively. And we will see when CNN can fail.

2. Systematic Experimentation in Machine Learning

How can we prove our hypothesis that CNN has the inductive bias that the model is translation invariant? We conduct extensive experiments in machine learning research (and other fields) to prove our hypothesis. In this context, systematic experimentation refers to running a series of experiments to prove our hypothesis. In this homework, we will study how to conduct systematic experimentation in machine learning.

Let's take a step back and think about 1) what our hypothesis is and 2) what experiments are needed to conduct to prove our hypothesis. The first question is easy. The hypothesis is that CNN has the inductive biases of locality and translational invariance. It is not enough to show that CNN performs better than MLP with the same number of parameters. Then, how do we design the experiments to prove our hypothesis? In this homework, we will design the experiments, conduct the experiments, analyze the results, and draw a conclusion.

python

[Copy code](#)

```
import numpy as np
import random
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torch.utils.data import Dataset
import torchvision
import torchvision.transforms as T
from torchvision.transforms import ToPILImage

from PIL import Image
from scipy.ndimage.interpolation import rotate
from sklearn.linear_model import LogisticRegression
from tqdm import tqdm
from copy import deepcopy
from torch.utils.data import DataLoader

class EdgeDetectionDataset(Dataset):
    def __init__(self, domain_config, mode="train", transform=None) -> None
        """
```


Args:

```
    domain_config (dict): Domain configuration
        data_per_class (int): Number of data per class
        num_classes (int): Number of classes
        class_type (list): List of class types
        spatial_resolution (int): length of height and width of the
        max_edge_width (int): Maximum edge width
        max_edge_intensity (float): Maximum edge intensity
        min_edge_intensity (float): Minimum edge intensity
        max_background_intensity (float): Maximum background intensity
        min_background_intensity (float): Minimum background intensity
        possible_edge_location_ratio (float): Confine the possible location
        num_horizontal_edge (int): Number of horizontal edges
        num_vertical_edge (int): Number of vertical edges
        use_permutation (bool): Whether to apply random permutation
    mode (str): Mode of the dataset (train, val, test)
    transform (callable, optional): Optional transform to be applied
    """
    self.data_per_class = domain_config.get("data_per_class", 1000)
    self.num_classes = domain_config.get("num_classes", 3)
    self.class_type = domain_config.get(
        "class_type", ["horizontal", "vertical", "none"]
    )
    self.spatial_resolution = domain_config.get("spatial_resolution", 256)
    self.min_edge_width = domain_config.get("min_edge_width", 1)
    self.max_edge_width = domain_config.get("max_edge_width", 4)
    self.max_edge_intensity = domain_config.get("max_edge_intensity", 1)
    self.min_edge_intensity = domain_config.get("min_edge_intensity", 0)
    self.max_background_intensity = domain_config.get(
        "max_background_intensity", 0.2
    )
    self.min_background_intensity = domain_config.get("min_background_intensity", 0)
    self.possible_edge_location_ratio = domain_config.get(
        "possible_edge_location_ratio", 1.0
    )
    self.num_horizontal_edge = domain_config.get("num_horizontal_edge", 1)
    self.num_vertical_edge = domain_config.get("num_vertical_edge", 1)
    self.num_diagonal_edge = domain_config.get("num_diagonal_edge", 1)
    self.use_permutation = domain_config.get("use_permutation", False)
    self.permutater = domain_config.get("permutater", None)
    self.unpermutater = domain_config.get("unpermutater", None)

    if self.possible_edge_location_ratio < 1.0:
        self.train_val_domain_shift = True
    else:
        self.train_val_domain_shift = False
```

```

self.possible_edge_location = int(
    self.possible_edge_location_ratio * self.spatial_resolution
)
self.mode = mode

assert self.num_classes == len(
    self.class_type
), "Number of classes must match the number of class types"

assert self.mode in (
    "train",
    "valid",
    "test",
), "Mode must be either train, valid, or test"

self.X = None
self.y = None

if self.use_permutation:
    assert self.permutater is not None, "permutater must be provided"
    assert self.unpermutater is not None, "Unpermutater must be provided"

self._generate_dataset()

self.transform = transform

def __len__(self):
    """
    Returns:
        int: Length of the dataset
    """
    return len(self.X)

def __getitem__(self, idx):
    """
    Args:
        idx (int): Index of the sample
    Returns:
        tuple: (sample, label)
    """
    if torch.is_tensor(idx):
        idx = idx.tolist()

    sample = self.X[idx]
    label = self.y[idx]

    if self.transform:
        - - - - -

```

```

        sample = self.transform(sample)

    return sample, label

def get_permutater(self):
    """
    Returns:
        np.ndarray: Permutation matrix
    """
    return self.permutater

def get_unpermutater(self):
    """
    Returns:
        np.ndarray: Unpermutation matrix
    """
    return self.unpermutater

def _permute_pixels(self, X):
    """
    Args:
        X (np.ndarray): Image
    Returns:
        np.ndarray: Permuted image
    """
    assert X.shape[0] == self.data_per_class, "Invalid image shape"
    assert len(X.shape) == 4, "Invalid image shape"

    n, h, w, c = X.shape

    X = X.reshape(n, h * w, c)
    X = X[:, self.permutater, :]
    X = X.reshape(n, h, w, c)

    return X

def _edge_intensity(self, edge_type="horizontal"):
    """
    Args:
        edge_type (str): Type of edge (horizontal, vertical, both, diagonal)
    Returns:
        np.ndarray: Edge intensity
    """
    if edge_type == "horizontal":
        num_edge = self.num_horizontal_edge
    elif edge_type == "vertical":
        num_edge = self.num_vertical_edge
    elif edge_type == "diagonal":
        num_edge = self.num_diagonal_edge
    elif edge_type == "both":
        num_edge = self.num_both_edge
    else:
        num_edge = 0
    return num_edge

```

```

elif edge_type == "diagonal":
    num_edge = self.num_diagonal_edge
elif edge_type == "both":
    num_edge = self.num_horizontal_edge + self.num_vertical_edge
else:
    raise ValueError("Invalid edge type")

return np.random.uniform(
    self.min_edge_intensity,
    self.max_edge_intensity,
    size=(self.data_per_class, num_edge),
)

def _edge_location(self, edge_type="horizontal"):
    """
    Args:
        edge_type (str): Type of edge (horizontal, vertical, both, diagonal)
    Returns:
        np.ndarray: Edge location
    """
    max_edge_width = self.max_edge_width + 1
    if edge_type == "horizontal":
        num_edge = self.num_horizontal_edge
    elif edge_type == "vertical":
        num_edge = self.num_vertical_edge
    elif edge_type == "diagonal":
        num_edge = self.num_diagonal_edge
        max_edge_width = int(self.max_edge_width / np.sqrt(2))
    elif edge_type == "both":
        num_edge = self.num_horizontal_edge + self.num_vertical_edge
    else:
        raise ValueError("Invalid edge type")

    edge_width = np.random.randint(
        self.min_edge_width, max_edge_width, size=(self.data_per_class, num_edge),
    )

    if self.mode == "train" and self.train_val_domain_shift:
        edge_location_start_idx = np.random.randint(
            0,
            self.possible_edge_location - edge_width,
            size=(self.data_per_class, num_edge),
        )
        edge_location_end_idx = np.clip(
            edge_location_start_idx + edge_width,
            0,
            self.possible_edge_location-1,
        )

```

```

    )

elif self.mode == "valid" and self.train_val_domain_shift:
    edge_location_start_idx = np.random.randint(
        self.possible_edge_location,
        self.spatial_resolution - edge_width,
        size=(self.data_per_class, num_edge),
    )
    edge_location_end_idx = np.clip(
        edge_location_start_idx + edge_width,
        self.possible_edge_location,
        self.spatial_resolution-1,
    )

else:
    edge_location_start_idx = np.random.randint(
        0,
        self.spatial_resolution - edge_width,
        size=(self.data_per_class, num_edge),
    )
    edge_location_end_idx = np.clip(
        edge_location_start_idx + edge_width,
        0,
        self.spatial_resolution-1,
    )

return edge_location_start_idx, edge_location_end_idx

def _generate_horizontal_edge_images(self):
    """
    Generate horizontal edge images
    Returns:
        np.ndarray: Generated horizontal edge images
    """
    assert (
        self.num_horizontal_edge > 0
    ), "Number of horizontal edge must be greater than 0"

    X = self._generate_background_images()

    edge_location_start_idx, edge_location_end_idx = self._edge_location(
        edge_type="horizontal"
    )
    edge_intensity = self._edge_intensity()

    for i in range(self.data_per_class):
        for j in range(self.num_horizontal_edge):
            vr

```

```

        ^L
        i, edge_location_start_idx[i, j] : edge_location_end_idx[
    ] = edge_intensity[i, j]

    return X

def _generate_vertical_edge_images(self):
    """
    Generate vertical edge images
    Returns:
        np.ndarray: Generated vertical edge images
    """
    assert (
        self.num_vertical_edge > 0
    ), "Number of vertical edge must be greater than 0"

    X = self._generate_background_images()

    edge_location_start_idx, edge_location_end_idx = self._edge_location
        edge_type="vertical"
    )
    edge_intensity = self._edge_intensity()

    for i in range(self.data_per_class):
        for j in range(self.num_vertical_edge):
            X[
                i,
                :,
                edge_location_start_idx[i, j] : edge_location_end_idx[i
                :,
                ] = edge_intensity[i, j]

    return X

def _generate_both_edge_images(self):
    """
    Generate horizontal/vertical edge images
    Returns:
        np.ndarray: Generated horizontal/vertical edge images
    """
    assert (
        self.num_horizontal_edge > 0
    ), "Number of horizontal edge must be greater than 0"
    assert (
        self.num_vertical_edge > 0
    ), "Number of vertical edge must be greater than 0"

    X = self._generate_background_images()

```

```

X = self._generate_background_images()

edge_location_start_idx, edge_location_end_idx = self._edge_location
    edge_type="both"
)
edge_intensity = self._edge_intensity(edge_type="both")

for i in range(self.data_per_class):
    for j in range(self.num_horizontal_edge):
        X[
            i,
            edge_location_start_idx[i, j] : edge_location_end_idx[i
            :,
            :,
            ] = edge_intensity[i, j]
    for j in range(self.num_vertical_edge):
        X[
            i,
            :,
            edge_location_start_idx[i, j] : edge_location_end_idx[i
            ] = edge_intensity[i, self.num_horizontal_edge + j]

return X

def _generate_diagonal_edge_images(self):
    """
    Generate diagonal edge images by rotating images
    Returns:
        np.ndarray: Generated diagonal edge images
    """
    assert (
        self.num_diagonal_edge > 0
    ), "Number of diagonal edge must be greater than 0"

    X = self._generate_background_images()
    background_intensity = np.mean(X, axis=(1, 2, 3))

    edge_location_start_idx, edge_location_end_idx = self._edge_location
        edge_type="diagonal"
    )
    edge_intensity = self._edge_intensity(edge_type="diagonal")

    random_angle = np.random.choice(
        [30, 45, 120, 135], size=(self.data_per_class, self.num_diagonal
    )

    for i in range(self.data_per_class):
        for j in range(self.num_diagonal_edge):

```

```

        for j in range(self.num_diagonal_edge):
            if i % 2 == 0: # horizontal
                X[
                    i,
                    edge_location_start_idx[i, j] : edge_location_end_idx[i, j],
                    :,
                    :,
                ] = edge_intensity[i, j]
            else: # vertical
                X[
                    i,
                    :,
                    edge_location_start_idx[i, j] : edge_location_end_idx[i, j],
                    :,
                ] = edge_intensity[i, j]
            X[i] = rotate(
                X[i],
                random_angle[i, j],
                reshape=False,
                mode="constant",
                cval=background_intensity[i],
            )
    return X

```

```

def _generate_background_images(self):
    """
    Generate background images
    Returns:
        np.ndarray: Generated background images
    """
    X = np.ones(
        (self.data_per_class, self.spatial_resolution, self.spatial_resolution)
    ) # NHWC format
    X *= np.random.uniform(
        self.min_background_intensity,
        self.max_background_intensity,
        size=(self.data_per_class, 1, 1, 1),
    )
    return X

```

```

def get_image_statistics(self):
    """
    Get image statistics
    Returns:
        tuple: (mean, std)
        mean (float): Mean of the images
        std (float): Standard deviation of the images
    """
    return self.mean, self.std

```



```

        return self._mean, self._std

def _generate_dataset(self):
    """
    Generate dataset
    Returns:
        tuple: (X, y)
        X (list of PIL Image): Generated images
        y (np.ndarray): Generated labels
    """
    num_data = self.data_per_class * self.num_classes
    self.X = np.zeros(
        (num_data, self.spatial_resolution, self.spatial_resolution, 1)
    )
    self.y = np.zeros((num_data,), dtype=np.int64)
    for i in range(self.num_classes):
        class_type = self.class_type[i]
        if class_type == "horizontal":
            X = self._generate_hoizontal_edge_images()
        elif class_type == "vertical":
            X = self._generate_vertical_edge_images()
        elif class_type == "both":
            X = self._generate_both_edge_images()
        elif class_type == "diagonal":
            X = self._generate_diagonal_edge_images()
        elif class_type == "none":
            X = self._generate_background_images()
        else:
            raise ValueError("Invalid class type")

        assert X.shape == (
            self.data_per_class,
            self.spatial_resolution,
            self.spatial_resolution,
            1,
        ) # NHWC format

        # permute pixels
        if self.use_permutation:
            X = self._permute_pixels(X)

        self.X[i * self.data_per_class : (i + 1) * self.data_per_class]
        self.y[i * self.data_per_class : (i + 1) * self.data_per_class]

    # Compute mean and std
    self._mean = np.mean(self.X)
    self._std = np.std(self.X)

```

```

# np.float32 -> np.uint8
self.X = (self.X * 255).astype(np.uint8)

# Convert ndarray to PIL Image
self.X = [T.functional.to_pil_image(x) for x in self.X]

def count_parameters(model, only_trainable=False):
    if only_trainable:
        return sum(p.numel() for p in model.parameters() if p.requires_grad)
    else:
        return sum(p.numel() for p in model.parameters())

def freeze_conv_layer(model):
    for name, param in model.named_parameters():
        if name.startswith('conv'):
            param.requires_grad = False

def init_conv_kernel_with_edge_detector(model):
    # Get kernel size
    kernel_size = model.conv1.kernel_size[0]

    # number of filters should be 3
    num_filters = model.conv1.out_channels
    assert num_filters == 3, "Number of filters should be 3"

    if kernel_size == 2:
        # 2 x 2 edge detector
        horizontal_edge_detector = torch.tensor([[1, 1], [-1, -1]], dtype=torch.float32)
        vertical_edge_detector = torch.tensor([[1, -1], [1, -1]], dtype=torch.float32)
        none_edge_detector = torch.tensor([[0, 0], [0, 0]], dtype=torch.float32)
    elif kernel_size == 7:
        # 7 x 7 edge detector
        horizontal_edge_detector = torch.tensor([
            [0, 0, 0, 0, 0, 0, 0],
            [0, 0, 0, 0, 0, 0, 0],
            [1, 1, 1, 1, 1, 1, 1],
            [-1, -1, -1, -1, -1, -1, -1],
            [0, 0, 0, 0, 0, 0, 0],
            [0, 0, 0, 0, 0, 0, 0],
            [0, 0, 0, 0, 0, 0, 0]], dtype=torch.float32)
        vertical_edge_detector = horizontal_edge_detector.T
        none_edge_detector = model.conv1.weight.data[0, 0] # torch.zeros((7, 7))
    else:
        horizontal_edge_detector = torch.from_numpy(custom_sobel((kernel_size, kernel_size)))
        vertical_edge_detector = torch.from_numpy(custom_sobel((kernel_size, kernel_size)))
        none_edge_detector = torch.from_numpy(np.zeros((kernel_size, kernel_size)))

```

```

edge_detector = torch.stack([horizontal_edge_detector, vertical_edge_detector])
model.conv1.weight.data = edge_detector.view(model.num_filter, 1, model.in_channels, model.conv1.kernel_size[0], model.conv1.kernel_size[1])
model.conv2.weight.data = torch.cat([model.conv1.weight.data, model.conv1.weight.data], 1)

```

```

# type casting

```

```

model.conv1.weight.data = model.conv1.weight.data.type(torch.FloatTensor)
model.conv2.weight.data = model.conv2.weight.data.type(torch.FloatTensor)

```

```

# bias

```

```

model.conv1.bias.data = torch.tensor([0, 0, 0], dtype=torch.float32)
model.conv2.bias.data = torch.tensor([0, 0, 0], dtype=torch.float32)

```

```

def custom_sobel(shape, axis):

```

```

    """

```

```

    shape must be odd: eg. (5,5)

```

```

    axis is the direction, with 0 to positive x and 1 to positive y

```

```

    """

```

```

    k = np.zeros(shape, dtype=np.float32)

```

```

    p = [(j,i) for j in range(shape[0])

```

```

          for i in range(shape[1])

```

```

          if not (i == (shape[1] - 1)/2. and j == (shape[0] - 1)/2.)]

```

```

    for j, i in p:

```

```

        j_ = int(j - (shape[0] - 1)/2.)

```

```

        i_ = int(i - (shape[1] - 1)/2.)

```

```

        k[j,i] = (i_ if axis==0 else j_)/float(i_*i_ + j_*j_)

```

```

    return k

```

```

def set_seed(seed):

```

```

    """

```

```

    Set the seed for all random number generators.

```

```

    """

```

```

    random.seed(seed)

```

```

    np.random.seed(seed)

```

```

    torch.manual_seed(seed)

```

```

    torch.cuda.manual_seed(seed)

```

```

    torch.cuda.manual_seed_all(seed)

```

```

def train_one_epoch(

```

```

    model,

```

```

    optimizer,

```

```

    criterion,

```

```

    train_loader,

```

```

    device,

```

```

epoch,
log_interval=100,
verbose=True,
):
    model.train()
    # return the average loss and accuracy
    train_loss = 0
    correct = 0

    for batch_idx, (data, target) in enumerate(train_loader):
        data, target = data.to(device), target.to(device)
        optimizer.zero_grad()
        output = model(data)
        loss = criterion(output, target)
        loss.backward()
        optimizer.step()

        train_loss += loss.item()
        pred = output.argmax(
            dim=1, keepdim=True
        )
        correct += pred.eq(target.view_as(pred)).sum().item()

    if batch_idx % log_interval == 0 and verbose:
        print(
            "Train Epoch: {} [{}/{} ({:.0f}%)]\tLoss: {:.6f}".format(
                epoch,
                batch_idx * len(data),
                len(train_loader.dataset),
                100.0 * batch_idx / len(train_loader),
                loss.item(),
            )
        )

    train_loss /= len(train_loader.dataset)
    train_accuracy = correct / len(train_loader.dataset) * 100.

    return train_loss, train_accuracy

def _generate_confusion_matrix(pred_list, target_list):
    pred_list = torch.cat(pred_list)
    target_list = torch.cat(target_list)

    assert pred_list.shape[0] == target_list.shape[0], "predictions and targets must have the same length"

    matrix_size = max(max(pred_list), max(target_list)) + 1
    confusion_matrix = torch.zeros(matrix_size, matrix_size)

```

```

for t, p in zip(target_list.view(-1), pred_list.view(-1)):
    confusion_matrix[t.long(), p.long()] += 1

return confusion_matrix.cpu().numpy()

def evaluate(model, criterion, valid_loader, device, verbose=True):
    model.eval()
    valid_loss = 0
    correct = 0

    pred_list, target_list = [], []
    confusion_matrix = torch.zeros(4, 4)

    with torch.no_grad():
        for data, target in valid_loader:
            data, target = data.to(device), target.to(device)
            output = model(data)
            valid_loss += criterion(output, target).item() # sum up batch
            pred = output.argmax(
                dim=1, keepdim=True
            ) # get the index of the max log-probability
            correct += pred.eq(target.view_as(pred)).sum().item()

            pred_list.append(pred)
            target_list.append(target)

    confusion_matrix = _generate_confusion_matrix(pred_list, target_list)

    valid_loss /= len(valid_loader.dataset)
    valid_accuracy = 100.0 * correct / len(valid_loader.dataset)

    if verbose:
        print(
            "Validation Result: Average loss: {:.4f}, Accuracy: {}/{:} ({:.0%}"
            " valid_loss, correct, len(valid_loader.dataset), valid_accuracy
        )

    return valid_loss, valid_accuracy, confusion_matrix

def vis_training_curve(cnn_train_loss, cnn_train_acc, mlp_train_loss, mlp_train_acc):
    # if mlp lists are empty, then we are only plotting the CNN
    if mlp_train_loss is None or len(mlp_train_loss) == 0:
        fig, ax = plt.subplots(1, 2, figsize=(20, 5))
        ax[0].plot(cnn_train_loss, label="CNN")

```

```

ax[0].set_title("Training Loss")
ax[0].set_xlabel("Epoch")
ax[0].set_ylabel("Loss")
ax[0].legend()
ax[0].grid()

ax[1].plot(cnn_train_acc, label="CNN")
ax[1].set_title("Training Accuracy")
ax[1].set_xlabel("Epoch")
ax[1].set_ylabel("Accuracy (%)")
ax[1].legend()
ax[1].grid()

plt.show()

# if cnn lists are empty, then we are only plotting the MLP
elif cnn_train_loss is None or len(cnn_train_loss) == 0:
    fig, ax = plt.subplots(1, 2, figsize=(20, 5))
    ax[0].plot(mlp_train_loss, label=label)
    ax[0].set_title("Training Loss")
    ax[0].set_xlabel("Epoch")
    ax[0].set_ylabel("Loss")
    ax[0].legend()
    ax[0].grid()

    ax[1].plot(mlp_train_acc, label=label)
    ax[1].set_title("Training Accuracy")
    ax[1].set_xlabel("Epoch")
    ax[1].set_ylabel("Accuracy (%)")
    ax[1].legend()
    ax[1].grid()

    plt.show()

# if both lists are not empty, then we are plotting both CNN and MLP
else:
    fig, ax = plt.subplots(1, 2, figsize=(20, 5))
    ax[0].plot(cnn_train_loss, label="CNN")
    ax[0].plot(mlp_train_loss, label=label)
    ax[0].set_title("Training Loss")
    ax[0].set_xlabel("Epoch")
    ax[0].set_ylabel("Loss")
    ax[0].legend()
    ax[0].grid()

    ax[1].plot(cnn_train_acc, label="CNN")
    ax[1].plot(mlp_train_acc, label=label)

```

```
ax[1].set_title("Training Accuracy")
ax[1].set_xlabel("Epoch")
ax[1].set_ylabel("Accuracy (%)")
ax[1].legend()
ax[1].grid()
```

```
plt.show()
```

```
def vis_validation_curve(cnn_valid_loss, cnn_valid_acc, mlp_valid_loss, mlp.
```

```
# if mlp lists are empty, then we are only plotting the CNN
```

```
if mlp_valid_loss is None or len(mlp_valid_loss) == 0:
```

```
    fig, ax = plt.subplots(1, 2, figsize=(20, 5))
```

```
    ax[0].plot(cnn_valid_loss, label="CNN")
```

```
    ax[0].set_title("Validation Loss")
```

```
    ax[0].set_xlabel("Epoch")
```

```
    ax[0].set_ylabel("Loss")
```

```
    ax[0].legend()
```

```
    ax[0].grid()
```

```
    ax[1].plot(cnn_valid_acc, label="CNN")
```

```
    ax[1].set_title("Validation Accuracy")
```

```
    ax[1].set_xlabel("Epoch")
```

```
    ax[1].set_ylabel("Accuracy (%)")
```

```
    ax[1].legend()
```

```
    ax[1].grid()
```

```
plt.show()
```

```
# if cnn lists are empty, then we are only plotting the MLP
```

```
elif cnn_valid_loss is None or len(cnn_valid_loss) == 0:
```

```
    fig, ax = plt.subplots(1, 2, figsize=(20, 5))
```

```
    ax[0].plot(mlp_valid_loss, label=label)
```

```
    ax[0].set_title("Validation Loss")
```

```
    ax[0].set_xlabel("Epoch")
```

```
    ax[0].set_ylabel("Loss")
```

```
    ax[0].legend()
```

```
    ax[0].grid()
```

```
    ax[1].plot(mlp_valid_acc, label=label)
```

```
    ax[1].set_title("Validation Accuracy")
```

```
    ax[1].set_xlabel("Epoch")
```

```
    ax[1].set_ylabel("Accuracy (%)")
```

```
    ax[1].legend()
```

```
    ax[1].grid()
```

```
plt.show()
```

```
# if both lists are not empty, then we are plotting both CNN and MLP
else:
```

```
    fig, ax = plt.subplots(1, 2, figsize=(20, 5))
    ax[0].plot(cnn_valid_loss, label="CNN")
    ax[0].plot(mlp_valid_loss, label=label)
    ax[0].set_title("Validation Loss")
    ax[0].set_xlabel("Epoch")
    ax[0].set_ylabel("Loss")
    ax[0].legend()
    ax[0].grid()
```

```
    ax[1].plot(cnn_valid_acc, label="CNN")
    ax[1].plot(mlp_valid_acc, label=label)
    ax[1].set_title("Validation Accuracy")
    ax[1].set_xlabel("Epoch")
    ax[1].set_ylabel("Accuracy (%)")
    ax[1].legend()
    ax[1].grid()
```

```
plt.show()
```

```
def vis_kernel(tensor, ch=0, allkernels=False, nrow=8, padding=1, title=None):
    n, c, h, w = tensor.shape
```

```
    if allkernels:
```

```
        tensor = tensor.view(n * c, -1, h, w)
```

```
    elif c != 3:
```

```
        tensor = tensor[:, ch, :, :].unsqueeze(dim=1)
```

```
    rows = np.min((tensor.shape[0] // nrow + 1, 64))
```

```
    grid = (
```

```
        torchvision.utils.make_grid(tensor, nrow=nrow, normalize=True, padding=padding)
        .numpy()
```

```
        .transpose((1, 2, 0))
```

```
)
```

```
plt.figure(figsize=(nrow, rows))
```

```
plt.imshow(grid, cmap=cmap)
```

```
# plt.colorbar(cmap=cmap)
```

```
if title is not None:
```

```
    plt.title(title, fontsize="small")
```

```
plt.axis("off")
```

```
plt.ioff()
```

```
plt.show()
```



```

def vis_confusion_matrix(confusion_matrix, class_names=None, title=None):
    fig = plt.figure(figsize=(10, 10))
    ax = fig.add_subplot(111)
    cax = ax.matshow(confusion_matrix, cmap=plt.cm.Blues)
    fig.colorbar(cax)

    matrix_size = confusion_matrix.shape[0]

    if class_names is not None:
        assert len(class_names) == matrix_size, "Class names must be same length as matrix size"
        ax.set_xticklabels([""] + class_names, rotation=90)
        ax.set_yticklabels([""] + class_names)

    ax.set_xlabel("Predicted")
    ax.set_ylabel("True")
    ax.xaxis.set_label_position("top")
    ax.xaxis.tick_top()
    ax.set_title(title)

    for (i, j), z in np.ndenumerate(confusion_matrix):
        ax.text(j, i, "{:0.1f}".format(z), ha="center", va="center")

def vis_unpermuted_dataset(dataset, num_classes, num_show_per_class, unpermuted, unpermuted_index):
    f, axarr = plt.subplots(num_classes, num_show_per_class, figsize=(20, 20))

    for i in range(num_classes):
        for j in range(num_show_per_class):
            img = dataset[i * num_show_per_class + j][0]
            label = dataset[i * num_show_per_class + j][1]

            if isinstance(img, torch.Tensor):
                img = img.numpy().transpose((1, 2, 0))
                h, w, c = img.shape
                assert c == 1
                img = img.reshape((h * w, c))
                img = img[unpermuted, :]
                img = img.reshape(h, w)
                axarr[i, j].imshow(img, cmap="gray", vmin=0, vmax=1)

            elif isinstance(img, Image.Image):
                img = np.array(img)
                h, w = img.shape
                img = img.reshape(h*w)
                img = img[unpermuted]
                img = img.reshape(h, w)
                img = T.functional.to_pil_image(img)

```

```

        axarr[i, j].imshow(img, cmap="gray", vmin=0, vmax=255)

    axarr[i, j].axis("off")
    axarr[i, j].set_title('Class: {}'.format(label))

def vis_dataset(dataset, num_classes=3, num_show_per_class=10):
    f, axarr = plt.subplots(num_classes, num_show_per_class, figsize=(20, 20))

    for i in range(num_classes):
        for j in range(num_show_per_class):
            img = dataset[i * num_show_per_class + j][0]
            label = dataset[i * num_show_per_class + j][1]

            if isinstance(img, torch.Tensor):
                img = img.numpy().transpose((1, 2, 0))
                img = img.squeeze()
                axarr[i, j].imshow(img, cmap="gray", vmin=0, vmax=1)
            elif isinstance(img, Image.Image):
                axarr[i, j].imshow(img, cmap="gray", vmin=0, vmax=255)
            axarr[i, j].axis("off")
            axarr[i, j].set_title('Class: {}'.format(label))

class WiderCNN(nn.Module):
    def __init__(self, input_channel=1, num_filters=6, kernel_size=7, num_classes=10):
        super(WiderCNN, self).__init__()
        padding = (kernel_size - 1) // 2
        self.conv1 = nn.Conv2d(input_channel, num_filters, kernel_size=kernel_size, padding=padding)
        self.conv2 = nn.Conv2d(num_filters, num_filters, kernel_size=kernel_size, padding=padding)
        self.maxpool = nn.MaxPool2d(2, 2)
        self.fc = nn.Linear(num_filters, num_classes)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        x = self.maxpool(x)
        x = F.relu(self.conv2(x))
        x = F.adaptive_avg_pool2d(x, (1, 1)).squeeze()
        x = self.fc(x)

    return x

class DeeperCNN(nn.Module):
    def __init__(self, input_channel=1, num_filters=3, kernel_size=7, num_classes=10):
        super().__init__()
        padding = (kernel_size - 1) // 2
        self.conv1 = nn.Conv2d(input_channel, num_filters, kernel_size=kernel_size, padding=padding)

```

```

self.conv2 = nn.Conv2d(num_filters, num_filters, kernel_size=kernel.
self.conv3 = nn.Conv2d(num_filters, num_filters, kernel_size=kernel.
self.conv4 = nn.Conv2d(num_filters, num_filters, kernel_size=kernel.
self.maxpool = nn.MaxPool2d(2, 2)
self.fc1 = nn.Linear(num_filters, num_classes)

```

```

self.num_filters = num_filters

```

```

def forward(self, x):
    x = F.relu(self.conv1(x))
    x = F.relu(self.conv2(x))
    x = self.maxpool(x)
    x = F.relu(self.conv3(x))
    x = F.relu(self.conv4(x))
    x = F.adaptive_avg_pool2d(x, (1, 1)).squeeze()
    x = self.fc1(x)
    return x

```

```

class SimpleCNN(nn.Module):

```

```

    def __init__(self, num_filters=3, kernel_size=2, num_classes=3):
        super().__init__()
        padding = (kernel_size - 1) // 2
        self.conv1 = nn.Conv2d(1, num_filters, kernel_size, padding=padding
        self.conv2 = nn.Conv2d(num_filters, num_filters, kernel_size, paddi
        self.maxpool = nn.MaxPool2d(2, 2)
        self.fc = nn.Linear(num_filters, num_classes)

```

```

        self.num_filter = num_filters
        self.kernel_size = kernel_size

```

```

    def forward(self, x):
        x = self.conv1(x)
        x = F.relu(x)
        x = self.maxpool(x)
        x = self.conv2(x)
        x = F.relu(x)
        x = F.adaptive_avg_pool2d(x, (1, 1)).squeeze()
        x = self.fc(x)

```

```

    return x

```

```

    def get_features(self, x):

```

```

        feat_list = []
        x = self.conv1(x)
        feat_list.append(x)
        x = F.relu(x)
        feat_list.append(x)
        _ _ _ _

```

```

x = self.maxpool(x)
feat_list.append(x)
x = self.conv2(x)
feat_list.append(x)
x = F.relu(x)
feat_list.append(x)
x = F.adaptive_avg_pool2d(x, (1, 1)).squeeze()
feat_list.append(x)

return feat_list

```

```

class SimpleCNN_avgpool(nn.Module):
    def __init__(self, num_filters=3, kernel_size=2, num_classes=3):
        super().__init__()
        padding = (kernel_size - 1) // 2
        self.conv1 = nn.Conv2d(1, num_filters, kernel_size, padding=padding)
        self.conv2 = nn.Conv2d(num_filters, num_filters, kernel_size, padding=padding)
        self.avgpool = nn.AvgPool2d(2, 2)
        self.fc = nn.Linear(num_filters, num_classes)

        self.num_filter = num_filters
        self.kernel_size = kernel_size

    def forward(self, x):
        x = self.conv1(x)
        x = F.relu(x)
        x = self.avgpool(x)
        x = self.conv2(x)
        x = F.relu(x)
        x = F.adaptive_avg_pool2d(x, (1, 1)).squeeze()
        x = self.fc(x)

        return x

    def get_features(self, x):
        feat_list = []
        x = self.conv1(x)
        feat_list.append(x)
        x = F.relu(x)
        feat_list.append(x)
        x = self.avgpool(x)
        feat_list.append(x)
        x = self.conv2(x)
        feat_list.append(x)
        x = F.relu(x)
        feat_list.append(x)

```

```

x = F.adaptive_avg_pool2d(x, (1, 1)).squeeze()
feat_list.append(x)

```

```

return feat_list

```

```

class ThreeLayerCNN(nn.Module):

```

```

    def __init__(

```

```

        self,
        input_dim=(1, 28, 28),
        num_filters=64, #make it explicit
        filter_size=7,
        hidden_dim=100,
        num_classes=4,

```

```

    ):

```

```

        """

```

```

        A three-layer convolutional network with the following architecture
        conv - relu - 2x2 max pool - affine - relu - affine - softmax
        The network operates on minibatches of data that have shape (N, C, H, W)
        consisting of N images, each with height H and width W and with C input
        channels.

```

```

        Args:

```

```

            kernel_size (int): Size of the convolutional kernel
            channel_size (int): Number of channels in the convolutional layer
            linear_layer_input_dim (int): Number of input features to the linear layer
            output_dim (int): Number of output features

```

```

        """

```

```

        super(ThreeLayerCNN, self).__init__()

```

```

        C, H, W = input_dim

```

```

        self.conv1 = nn.Conv2d(

```

```

            C, num_filters, filter_size, stride=1, padding=(filter_size - 1) // 2
        )

```

```

        self.max_pool = nn.MaxPool2d(2, 2)

```

```

        self.conv2 = nn.Conv2d(

```

```

            num_filters, num_filters * 2, filter_size, padding=(filter_size - 1) // 2
        )

```

```

        self.fc1 = nn.Linear(num_filters * 2, num_classes)

```

```

    def forward(self, x):

```

```

        x = F.relu(self.conv1(x))

```

```

        x = F.relu(self.conv2(x))

```

```

        x = F.adaptive_avg_pool2d(x, (1, 1)).squeeze()

```

```

        x = self.fc1(x)

```

```

        return x

```

```

class TwoLayerMLP(nn.Module):

```

```

    def __init__(self, input_dim=64, hidden_dim=64, num_classes=10):

```

```

def __init__(self, input_dim=(1, 28, 28), hidden_dim=10, num_classes=3):
    super(TwoLayerMLP, self).__init__()
    C, H, W = input_dim
    self.fc1 = nn.Linear(C * H * W, hidden_dim)
    self.fc2 = nn.Linear(hidden_dim, num_classes)

def forward(self, x):
    x = x.view(x.size(0), -1)
    x = F.relu(self.fc1(x))
    x = self.fc2(x)
    return x

class ThreeLayerMLP(nn.Module):
    def __init__(self, input_dim=(1, 28, 28), hidden_dims=[10, 10], num_classes=3):
        """
        A three-layer fully-connected neural network with ReLU nonlinearity
        """
        super(ThreeLayerMLP, self).__init__()
        torch.manual_seed(seed)
        C, H, W = input_dim
        self.fc1 = nn.Linear(C * H * W, hidden_dims[0])
        self.fc2 = nn.Linear(hidden_dims[0], hidden_dims[1])
        self.fc3 = nn.Linear(hidden_dims[1], num_classes)

    def forward(self, x):
        x = x.view(x.size(0), -1)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)

        return x

# Define the domain configuration of the dataset
set_seed(seed)

visualize_data_config = dict(
    data_per_class=10,
    num_classes=3,
    class_type=["horizontal", "vertical", "none"],
)

visualize_dataset = EdgeDetectionDataset(visualize_data_config, mode='train

```

Llama

It seems you've provided a large code snippet that appears to be related to a homework

assignment or a deep learning project. The code defines several classes and functions for creating and training neural networks, specifically convolutional neural networks (CNNs) and multi-layer perceptrons (MLPs), on an edge detection task.

To proceed, I'd like to know what specific problem or task you'd like me to help with. Are you:

1. Trying to debug an issue with the code?
2. Looking to modify the code to achieve a specific goal (e.g., change the architecture, add a new feature)?
3. Seeking an explanation of a particular concept or technique used in the code?
4. Needing help with implementing a new functionality or experiment?

Please provide more context or clarify your question, and I'll do my best to assist you.

anonymous

Q1. Overfitting Models to Small Dataset

In this problem, we will make our models overfit the small dataset to test the model architecture and our synthetic dataset. We use the same dataset for both models. Let's generate a small dataset with ten images per class.

In this notebook, we will use pytorch dataloader to load the dataset. We will use `torch.utils.data.DataLoader` to load the dataset. `DataLoader` takes two arguments: `dataset` and `batch_size`. `dataset` is the dataset that we want to load. Note that `batch_size` is one of important hyperparameters. We will use `batch_size=32` for this problem.

shell

Copy code

```
set_seed(seed)
```

```
small_dataset_config = None
small_dataset = None
transforms = T.Compose([T.ToTensor()])
```

```
#####
# TODO: Generate dataset with 10 images per class
# Hint: Refer visualize_data_config and use EdgeDetectionDataset and
# transforms function provided above.
#####
#####
```

```
#                                END OF YOUR CODE
#####;

shell                                                                    Copy code

small_dataset_loader = None

#####;
# TODO: Implement dataloader
# Hint: You should flag shuffle = True for training data loader
# This flag makes huge difference in training
#####;
#####;
#                                END OF YOUR CODE
#####;
```

Model Architecture

MLP has two hidden layer with 10 hidden units and 10 hidden units. The input size is $28 \times 28 = 784$ and the output size is 3. We use ReLU as the activation function. We use cross entropy loss as the loss function.

MLP architecture: $FC(784, 10) \rightarrow ReLU \rightarrow FC(10, 10) \rightarrow ReLU \rightarrow FC(10, 3)$

CNN has two convolutional layers followed by global average pooling and one fully connected layer. Both convolutional layers have 3 filters whose kernel size is 7. We use ReLU as the activation function. We use cross entropy loss as the loss function.

CNN arhitecture is as follows: CONV - RELU - MAXPOOL - CONV - RELU - MAXPOOL - FC

Llama

To complete the task, we will follow the given instructions and fill in the required code.

Step 1: Generate a small dataset with 10 images per class

```
python                                                                    Copy code

set_seed(seed)

small_dataset_config = dict(
    data_per_class=10,
    num_classes=3,
    class_type=["horizontal", "vertical", "none"],
)
```



```
small_dataset = EdgeDetectionDataset(small_dataset_config, mode='train', tr
```

Step 2: Implement the dataloader

python

[Copy code](#)

```
small_dataset_loader = torch.utils.data.DataLoader(small_dataset, batch_size
```

The `DataLoader` is initialized with the `small_dataset`, a batch size of 32, and `shuffle=True` to randomize the order of the samples.

The completed code is as follows:

python

[Copy code](#)

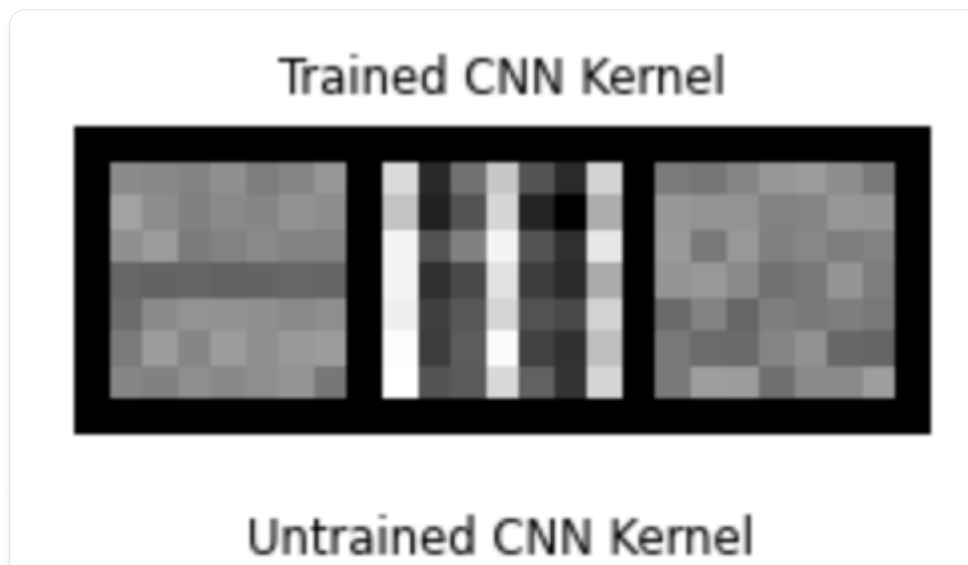
```
set_seed(seed)
```

```
small_dataset_config = dict(  
    data_per_class=10,  
    num_classes=3,  
    class_type=["horizontal", "vertical", "none"],  
)
```

```
small_dataset = EdgeDetectionDataset(small_dataset_config, mode='train', tr
```

```
small_dataset_loader = torch.utils.data.DataLoader(small_dataset, batch_size
```

anonymous





Question

Can you find any interesting patterns in the learned filters?

Llama

The image shows the kernels (or filters) of a Convolutional Neural Network (CNN) before and after training. The trained CNN kernel is displayed at the top, and the untrained CNN kernel is shown at the bottom.

Upon examining the trained CNN kernel, we can observe the following:

- The first filter appears to be detecting a horizontal edge or a smooth gradient.
- The second filter seems to be detecting a vertical edge.
- The third filter does not have an obvious pattern.

In contrast, the untrained CNN kernel displays random weights, which is typical for the initial state of a neural network before training.

Interesting patterns:

1. **Edge detection:** The trained CNN kernel has filters that resemble edge detectors, which is a common phenomenon in CNNs trained on image data. The second filter, in particular, is a clear example of a vertical edge detector.
2. **Learning meaningful representations:** The trained CNN kernel has learned meaningful representations from the data, as evident from the structured patterns in the filters. This suggests that the CNN has learned to extract useful features from the input data.
3. **Comparison with untrained kernel:** The contrast between the trained and untrained kernels highlights the impact of training on the CNN's ability to learn relevant features. The untrained kernel appears random, while the trained kernel displays more structured and interpretable patterns.

Overall, the learned filters in the trained CNN kernel demonstrate the network's ability to extract meaningful features from the input data, which is a key aspect of its functionality.

anonymous

Q2. Sweeping the Number of Training Images

We understood the given task and checked that both models had enough expressive power. We will compare the performance of MLP and CNN by changing the number of data per class. We expect that the model with proper inductive biases on this task will fit with **fewer training examples**. And let's see which one has inductive biases. In this problem, we will use the same dataset for both models. We sweep the number of training images from 10 to 500. The validation set will be the same for all the experiments.

makefile

Copy code

```
set_seed(seed)

train_loader_dict = dict()
num_images_list = [10, 30, 50, 100]
valid_loader = None

transforms = T.Compose([T.ToTensor()])
train_batch_size = 10
valid_batch_size = 256
#####;
# TODO: Implement train_loader_dict for each number of training images.
# Key: The number of training images (5, 10, 30, 50, and 100)
# Value: The corresponding dataloader
# The validation set size is 50 images per class
#####;
#####;
#
#                               END OF YOUR CODE
#####;
```

Llama

To implement the `train_loader_dict` for each number of training images and the `valid_loader`, we can follow these steps:

1. Define the dataset configurations for training and validation.

2. Create the datasets using the `EdgeDetectionDataset` class.
3. Initialize the dataloaders using `torch.utils.data.DataLoader`.

Here's the code to achieve this:

python

[Copy code](#)

```
set_seed(seed)

train_loader_dict = dict()
num_images_list = [10, 30, 50, 100]
valid_loader = None

transforms = T.Compose([T.ToTensor()])
train_batch_size = 10
valid_batch_size = 256

# Define the dataset configurations
dataset_config = dict(
    num_classes=3,
    class_type=["horizontal", "vertical", "none"],
)

# Create the validation dataset and dataloader
valid_dataset_config = deepcopy(dataset_config)
valid_dataset_config['data_per_class'] = 50
valid_dataset = EdgeDetectionDataset(valid_dataset_config, mode='valid', tra
valid_loader = torch.utils.data.DataLoader(valid_dataset, batch_size=valid_l

# Create the training datasets and dataloaders for different numbers of tra
for num_images in num_images_list:
    train_dataset_config = deepcopy(dataset_config)
    train_dataset_config['data_per_class'] = num_images
    train_dataset = EdgeDetectionDataset(train_dataset_config, mode='train'
    train_loader_dict[num_images] = torch.utils.data.DataLoader(train_data
```

This code creates the `train_loader_dict` with dataloaders for different numbers of training images and the `valid_loader` for the validation set. The validation set size is fixed at 50 images per class.

anonymous

Trained Kernel - data: 10



Untrained Kernel - data: 10



Trained Kernel - data: 30



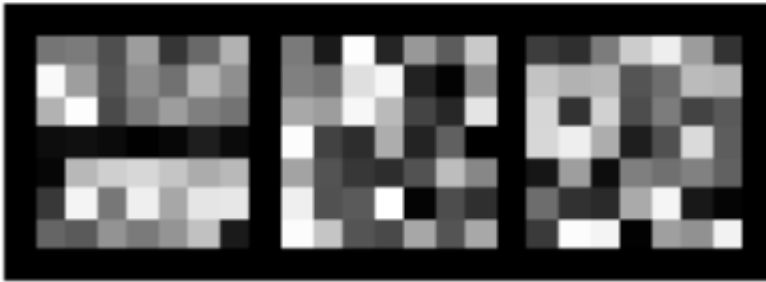
Untrained Kernel - data: 30



Trained Kernel - data: 50



Untrained Kernel - data: 50



Trained Kernel - data: 100



Untrained Kernel - data: 100



Question

Compare the learned kernels, untrainable kernels, and edge-detector kernels. What do you observe? Answer this question in your submission of the written assignment.

Llama

Observations

The image displays a series of kernel visualizations for a Convolutional Neural Network (CNN) trained on different amounts of data (10, 30, 50, and 100 samples). Each section shows the trained kernel and the untrained kernel side by side.

Upon examining the trained kernels, we can observe the following:

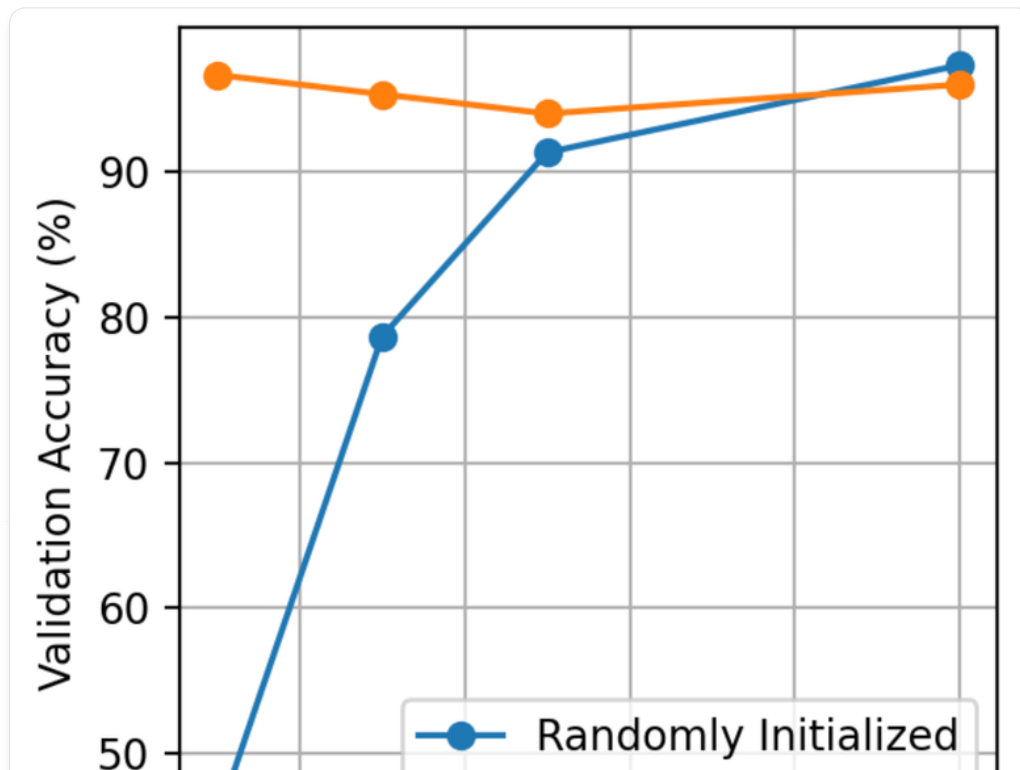
- **Edge detection:** The trained kernels exhibit a clear edge detection pattern, particularly for the vertical edge. This is evident from the second filter in each trained kernel visualization, which displays a strong vertical gradient.
- **Increasing data:** As the amount of training data increases (from 10 to 100), the trained kernels become more refined and resemble the edge-detector kernels more closely.
- **Comparison with untrained kernels:** The untrained kernels appear random and lack the structured patterns observed in the trained kernels.

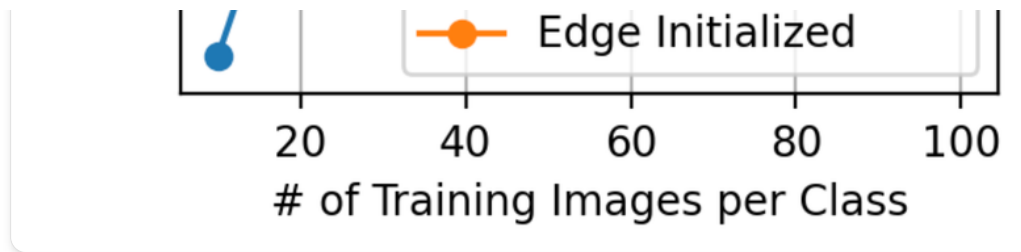
Key takeaways:

1. The CNN learns to detect edges as it trains on more data.
2. The trained kernels become more similar to edge-detector kernels as the training dataset size increases.
3. The untrained kernels lack the structured patterns seen in the trained kernels, indicating that the training process is essential for learning meaningful representations.

These observations suggest that the CNN is learning to extract relevant features (edges) from the input data, and the quality of these features improves with more training data.

anonymous





Question

We freeze the convolutional layer and train only final layer (classifier) in this experiment. For a high data regime, the performance of CNN initialized with edge detectors is worse than CNN initialized with random weights. **Why do you think this happens?** Answer this question in your submission of the written assignment.

Llama

Possible Explanation

The graph shows that when the number of training images per class is low (less than 50), the CNN initialized with edge detectors performs better than the CNN initialized with random weights. However, as the number of training images per class increases (beyond 50), the performance of the CNN initialized with random weights surpasses that of the CNN initialized with edge detectors.

One possible explanation for this phenomenon is that when the convolutional layer is frozen, the features extracted by the convolutional layer are not adapted to the specific task during training. In the case of the CNN initialized with edge detectors, the convolutional layer is initialized with a specific set of features (edge detectors) that are relevant to the task. However, these features may not be optimal for the task, and freezing the convolutional layer prevents the network from adapting these features during training.

In contrast, the CNN initialized with random weights has the potential to learn more diverse and task-specific features during training if the convolutional layer were not frozen. Although the convolutional layer is frozen in this experiment, the fact that it is randomly initialized means that it has not been constrained to a specific set of features (edge detectors). When there is a large amount of training data, the final layer (classifier) can still learn to make accurate predictions using the frozen features.

However, it's more likely that the issue here is that the edge detectors, although a good initialization, might be too simplistic or not flexible enough to capture the nuances of the data when there's a lot of it. When there's a lot of training data, the randomly initialized CNN has the capacity to learn more complex and nuanced features that can lead to better performance.

Overfitting or underfitting of the edge detectors to the task: Another possible explanation is that the edge detectors are a good prior for the task when there's limited data, but they might not be the optimal solution when there's a lot of data. The randomly initialized CNN can potentially learn a more optimal representation of the data.

Inductive bias: The edge detectors impose a strong inductive bias on the network, which can be beneficial when there's limited data. However, when there's a lot of data, this bias might become a limitation, and the network may not be able to learn the optimal representation.

Overall, the results suggest that while initializing the CNN with edge detectors can be beneficial in low-data regimes, it may not be the best approach when there's a large amount of training data.

This is a really detailed solution and covers all possible reasons!

anonymous

Previously we used 30 epochs. We currently get 75% accuracy for CNN in 30 images class

css

Copy code

```
#####
# TODO: Try other num_epochs. Final accuracy of CNN should be at around
# 95-99% for 30 images per class.
#####
#####
#
#                               END OF YOUR CODE
#####
lr = 1e-2
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
criterion = nn.CrossEntropyLoss()

cnn_acc_list = list()
mlp_acc_list = list()

cnn_kernel_dict = dict()
untrained_cnn_kernel_dict = dict()

for num_image, train_loader in train_loader_dict.items():
    print("Training with {} images".format(num_image))
    set_seed(seed)
    cnn_model = SimpleCNN(kernel_size=7)
    untrained_cnn_model = deepcopy(cnn_model)
    cnn_model.to(device)
```

```

mlp_model = ThreeLayerMLP(hidden_dims=[50, 10])
mlp_model.to(device)

mlp_optimizer = optim.SGD(mlp_model.parameters(), lr=lr, momentum=0.9)
cnn_optimizer = optim.SGD(cnn_model.parameters(), lr=lr, momentum=0.9)

mlp_results = train_model(mlp_model, mlp_optimizer, num_epochs, train_loader, dev_loader)
cnn_results = train_model(cnn_model, cnn_optimizer, num_epochs, train_loader, dev_loader)

vis_training_curve(cnn_results["train_loss"], cnn_results["train_acc"],
vis_validation_curve(cnn_results["valid_loss"], cnn_results["valid_acc"])

cnn_kernel_dict[num_image] = deepcopy(cnn_model.conv1.weight.data.detach())
untrained_cnn_kernel_dict[num_image] = deepcopy(untrained_cnn_model.conv1.weight.data.detach())

print("CNN Acc: {}, MLP Acc: {}".format(cnn_results["final_valid_acc"], mlp_results["final_valid_acc"]))

```

Llama

To achieve a final accuracy of 95-99% for the CNN with 30 images per class, we need to adjust the `num_epochs` parameter. Let's try increasing the number of epochs.

python

Copy code

```

num_epochs = 100 # or some other value that achieves the desired accuracy

# Rest of the code remains the same

```

By increasing `num_epochs` to a larger value (e.g., 100), we allow the model to train for a longer period, which can help improve its accuracy.

Here's the updated code:

python

Copy code

```

lr = 1e-2
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
criterion = nn.CrossEntropyLoss()

cnn_acc_list = list()
mlp_acc_list = list()

cnn_kernel_dict = dict()

```

```

untrained_cnn_kernel_dict = dict()

num_epochs = 100

for num_image, train_loader in train_loader_dict.items():
    print("Training with {} images".format(num_image))
    set_seed(seed)
    cnn_model = SimpleCNN(kernel_size=7)
    untrained_cnn_model = deepcopy(cnn_model)
    cnn_model.to(device)

    mlp_model = ThreeLayerMLP(hidden_dims=[50, 10])
    mlp_model.to(device)

    mlp_optimizer = optim.SGD(mlp_model.parameters(), lr=lr, momentum=0.9)
    cnn_optimizer = optim.SGD(cnn_model.parameters(), lr=lr, momentum=0.9)

    mlp_results = train_model(mlp_model, mlp_optimizer, num_epochs, train_loader)
    cnn_results = train_model(cnn_model, cnn_optimizer, num_epochs, train_loader)

    vis_training_curve(cnn_results["train_loss"], cnn_results["train_acc"],
                      vis_validation_curve(cnn_results["valid_loss"], cnn_results["valid_acc"]

    cnn_kernel_dict[num_image] = deepcopy(cnn_model.conv1.weight.data.detach())
    untrained_cnn_kernel_dict[num_image] = deepcopy(untrained_cnn_model.conv1.weight.data.detach())

    print("CNN Acc: {}, MLP Acc: {}".format(cnn_results["final_valid_acc"],

```

You can adjust `num_epochs` to find the optimal value that achieves the desired accuracy for the CNN.

anonymous

Training with 10- CNN Acc: 72.0, MLP Acc: 50.0

Training with 30- CNN Acc: 99.33333333333333, MLP Acc: 71.33333333333333

Training with 50- CNN Acc: 98.66666666666667, MLP Acc: 87.33333333333333

Training with 100- CNN Acc: 100.0, MLP Acc: 96.66666666666667

Question

How much more data is needed for MLP to get a competitive performance with CNN?

Answer this question in your submission of the written assignment.

Llama

To determine how much more data is needed for the MLP to achieve a competitive performance with the CNN, let's analyze the given results:

- For 10 images per class, the CNN achieves 72% accuracy, while the MLP achieves 50% accuracy.
- For 30 images per class, the CNN achieves 99.33% accuracy, while the MLP achieves 71.33% accuracy. The CNN significantly outperforms the MLP.
- For 50 images per class, the CNN achieves 98.67% accuracy, while the MLP achieves 87.33% accuracy. The CNN still outperforms the MLP.
- For 100 images per class, the CNN achieves 100% accuracy, while the MLP achieves 96.67% accuracy. The MLP is now close to the CNN's performance.

Based on these results, we can observe that:

- The CNN consistently outperforms the MLP across different data sizes.
- The MLP's performance improves as the amount of data increases, but it lags behind the CNN's performance until there's a sufficient amount of data (100 images per class).

To achieve a competitive performance with the CNN, the MLP needs a significant amount of data. Based on the given results, it appears that the MLP requires around 100 images per class to achieve an accuracy close to the CNN's performance (96.67% vs 100%).

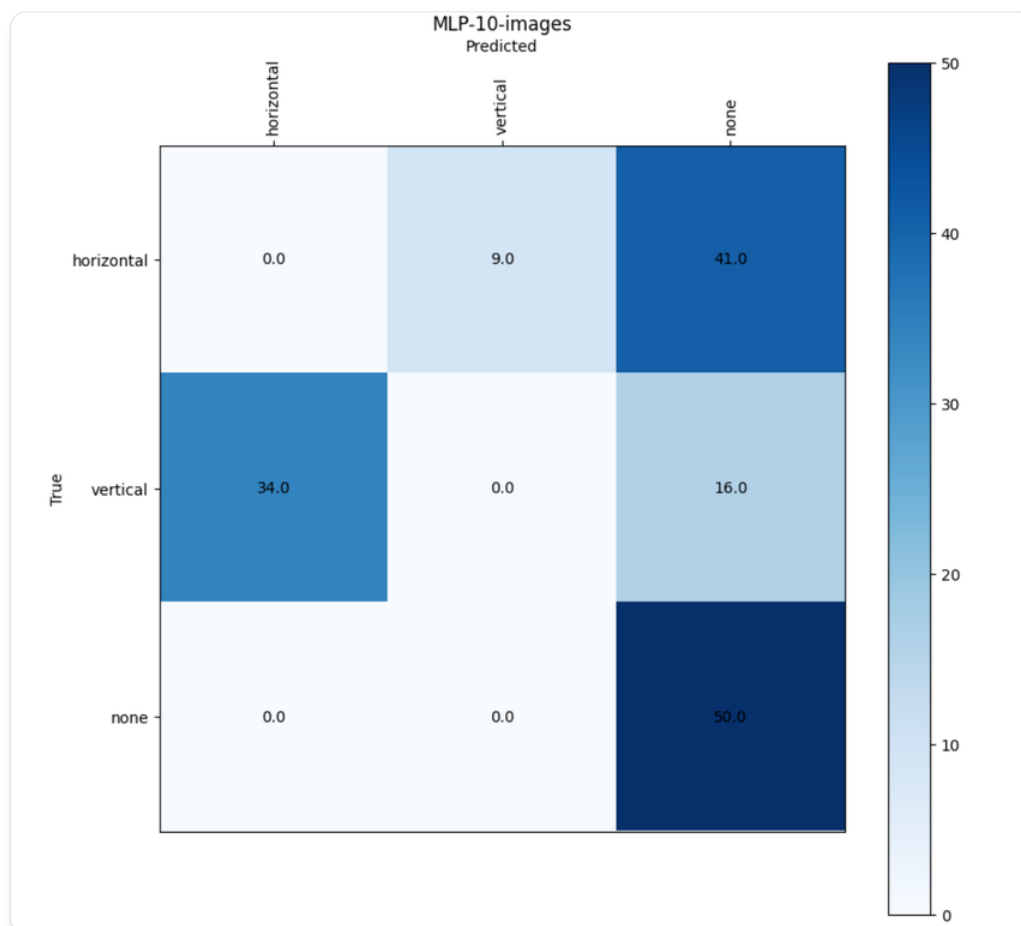
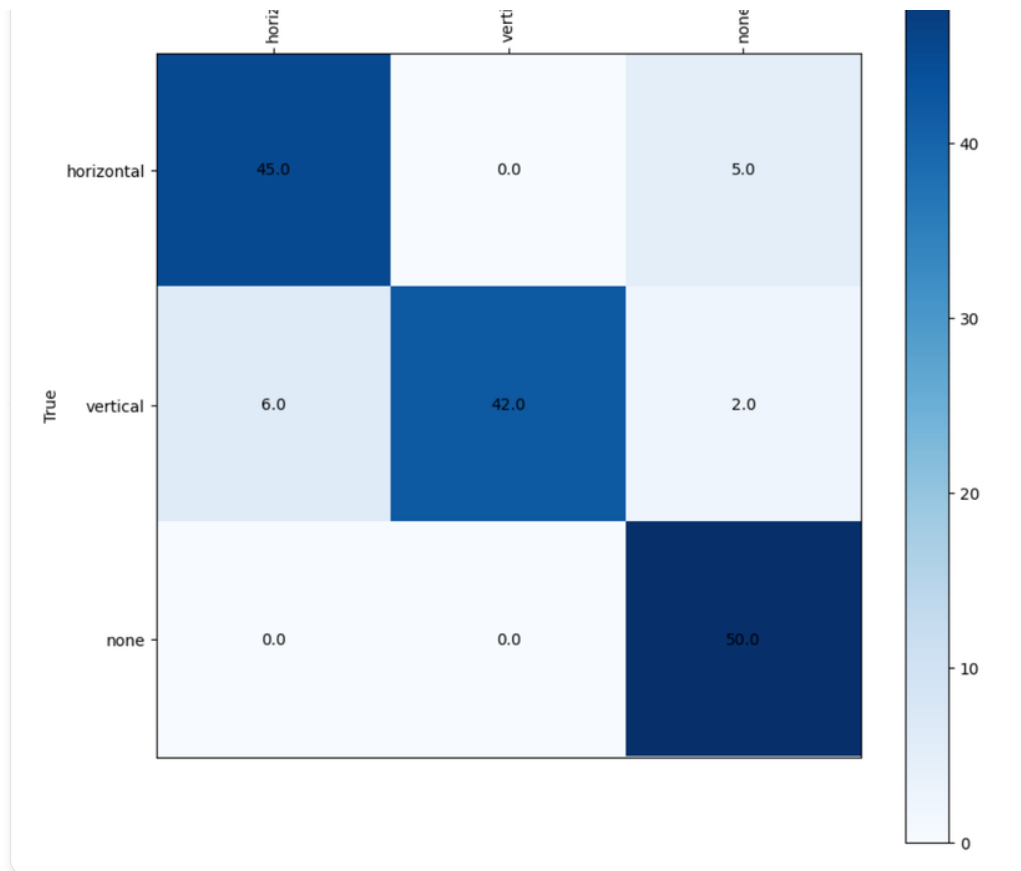
Therefore, for the MLP to get a competitive performance with the CNN, it would need:

- At least 100 images per class, as seen in the results.
- Possibly more data, as the MLP's accuracy is still slightly lower than the CNN's accuracy at 100 images per class.

In conclusion, the MLP needs substantially more data to match the CNN's performance. The exact amount of additional data required would depend on the specific requirements for "competitive performance." However, based on the given results, 100 images per class is a rough estimate.

anonymous





Q4. Domain Shift between Training and Validation Set

In this problem, we will see how the model performance changes when the domain of the training set and that of the validation set are different. We will generate training set images with edges that locate only half of the image and validation set images with edges that locate only the other half of the image. Let's repeat the same experiment as the previous problem.

Question

Why do you think the confusion matrix looks like this? Answer this question in your submission of the written assignment.

Llama

Confusion Matrix Analysis

The given confusion matrices represent the performance of a CNN and an MLP on a classification task with 10 images per class. The matrices display the true labels on the y-axis and the predicted labels on the x-axis.

CNN Confusion Matrix:

- The CNN correctly classifies most of the "horizontal" and "vertical" images (45 and 42, respectively).
- There is some confusion between "horizontal" and "none" (5 misclassifications) and between "vertical" and "none" (2 misclassifications) or "horizontal" (6 misclassifications).

MLP Confusion Matrix:

- The MLP struggles to correctly classify "horizontal" and "vertical" images, often misclassifying them as "none" (41 and 16, respectively).
- The MLP correctly classifies "none" images (50).

Comparison and Insights:

- The CNN outperforms the MLP in terms of correctly classifying "horizontal" and "vertical" images.
- The MLP tends to misclassify "horizontal" and "vertical" images as "none," indicating a potential bias towards the "none" class.

Domain Shift:

The training set images have edges that locate only half of the image, while the

validation set images have edges that locate the other half. This domain shift can cause the models to perform poorly on the validation set, as they are not trained on images with edges in the same location.

The confusion matrices suggest that both models are struggling with the domain shift. The CNN is still able to correctly classify some "horizontal" and "vertical" images, but it is not perfect. The MLP is heavily biased towards the "none" class, which may be due to the fact that it is not able to effectively capture the edge information in the images.

In summary, the confusion matrices indicate that the CNN is more robust to the domain shift than the MLP, but both models are affected by the difference in edge locations between the training and validation sets.

anonymous

python

[Copy code](#)

```
#####
# TODO: Try other num_epochs, lr, kernel_size. The validation accuracy
# should achieve around 97-100% for 10 images per class.
#####
#####
#
#                               END OF YOUR CODE
#####
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
criterion = nn.CrossEntropyLoss()

cnn_valid_acc_list = list()

cnn_kernel_dict = dict()

cnn_confusion_matrix_dict = dict()

for num_image, train_loader in train_loader_dict.items():
    print("Training with {} images".format(num_image))
    set_seed(seed)
    cnn_model = SimpleCNN(kernel_size=kernel_size)
    untrained_cnn_model = deepcopy(cnn_model)
    cnn_model.to(device)

    cnn_optimizer = optim.SGD(cnn_model.parameters(), lr=lr, momentum=0.9)

    cnn_results = train_model(cnn_model, cnn_optimizer, num_epochs, train_loader)

    vis_training_curve(cnn_results["train_loss"], cnn_results["train_acc"],
```

```
vis_validation_curve(cnn_results["valid_loss"], cnn_results["valid_acc"],  
                    print("CNN Acc: {}".format(cnn_results["final_valid_acc"])))
```

Llama

To achieve a validation accuracy of 97-100% for 10 images per class, we need to adjust the hyperparameters `num_epochs`, `lr`, and `kernel_size`. Let's try different combinations of these hyperparameters.

python

Copy code

```
num_epochs = 200 # increased number of epochs  
lr = 1e-2 # learning rate  
kernel_size = 7 # kernel size
```

```
# Rest of the code remains the same
```

Here's the updated code:

In this section, the LLM was not able to produce the correct output despite giving it hints. This is probably because it requires more guesswork and it did not make extreme guesses.

python

Copy code

```
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")  
criterion = nn.CrossEntropyLoss()
```

```
cnn_valid_acc_list = list()
```

```
cnn_kernel_dict = dict()
```

```
cnn_confusion_matrix_dict = dict()
```

```
num_epochs = 200  
lr = 1e-2  
kernel_size = 7
```

```
for num_image, train_loader in train_loader_dict.items():  
    print("Training with {} images".format(num_image))  
    set_seed(seed)  
    cnn_model = SimpleCNN(kernel_size=kernel_size)  
    untrained_cnn_model = deepcopy(cnn_model)  
    cnn_model.to(device)
```

```
cnn_optimizer = optim.SGD(cnn_model.parameters(), lr=lr, momentum=0.9)
```



```

cnn_results = train_model(cnn_model, cnn_optimizer, num_epochs, train_loader,
                           vis_training_curve, vis_validation_curve,
                           vis_validation_curve, cnn_results["valid_acc"])

print("CNN Acc: {}".format(cnn_results["final_valid_acc"]))

```

You can further tune these hyperparameters to achieve the desired validation accuracy. Some other possible combinations to try:

- num_epochs = 100, lr = 5e-3, kernel_size = 5
- num_epochs = 150, lr = 1e-2, kernel_size = 3
- num_epochs = 250, lr = 5e-2, kernel_size = 7

Experiment with different hyperparameter settings to find the optimal configuration for your task.

anonymous

You got CNN Acc: 72.66666666666667. Try again

Llama

Let's try again with a different set of hyperparameters.

python

Copy code

```

num_epochs = 250 # increased number of epochs
lr = 5e-2 # learning rate
kernel_size = 5 # kernel size

# Rest of the code remains the same

```

Here's the updated code:

python

Copy code

```

device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
criterion = nn.CrossEntropyLoss()

cnn_valid_acc_list = list()

```

```

cnn_kernel_dict = dict()

cnn_confusion_matrix_dict = dict()

num_epochs = 250
lr = 5e-2
kernel_size = 5

for num_image, train_loader in train_loader_dict.items():
    print("Training with {} images".format(num_image))
    set_seed(seed)
    cnn_model = SimpleCNN(kernel_size=kernel_size)
    untrained_cnn_model = deepcopy(cnn_model)
    cnn_model.to(device)

    cnn_optimizer = optim.SGD(cnn_model.parameters(), lr=lr, momentum=0.9)

    cnn_results = train_model(cnn_model, cnn_optimizer, num_epochs, train_loader)

    vis_training_curve(cnn_results["train_loss"], cnn_results["train_acc"],
                      vis_validation_curve(cnn_results["valid_loss"], cnn_results["valid_acc"],
                      plot_training_curve, plot_validation_curve))

    print("CNN Acc: {}".format(cnn_results["final_valid_acc"]))

```

This time, we have:

- num_epochs = 250
- lr = 5e-2
- kernel_size = 5

Let's see if this combination achieves the desired validation accuracy.

anonymous

You got CNN Acc: 58.666666666666664. Try again

Llama

Let's try again with a different set of hyperparameters.

python

Copy code

```

num_epochs = 300 # increased number of epochs
lr = 1e-1 # decreasing lr

```

```
lr = 1e-1 # learning rate
kernel_size = 3 # kernel size

# Rest of the code remains the same
```

Here's the updated code:

python

[Copy code](#)

```
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
criterion = nn.CrossEntropyLoss()

cnn_valid_acc_list = list()

cnn_kernel_dict = dict()

cnn_confusion_matrix_dict = dict()

num_epochs = 300
lr = 1e-1
kernel_size = 3

for num_image, train_loader in train_loader_dict.items():
    print("Training with {} images".format(num_image))
    set_seed(seed)
    cnn_model = SimpleCNN(kernel_size=kernel_size)
    untrained_cnn_model = deepcopy(cnn_model)
    cnn_model.to(device)

    cnn_optimizer = optim.SGD(cnn_model.parameters(), lr=lr, momentum=0.9)

    cnn_results = train_model(cnn_model, cnn_optimizer, num_epochs, train_loader)

    vis_training_curve(cnn_results["train_loss"], cnn_results["train_acc"],
                      vis_validation_curve(cnn_results["valid_loss"], cnn_results["valid_acc"]

    print("CNN Acc: {}".format(cnn_results["final_valid_acc"])))
```

This time, we have:

- num_epochs = 300
- lr = 1e-1
- kernel_size = 3

Let's see if this combination achieves the desired validation accuracy.

anonymous

Also failed. Try more epochs and slightly slower learning rate

Again, I tried many times to push it towards the right solution but it fails.

Llama

Let's try again with more epochs and a slightly slower learning rate.

python

Copy code

```
num_epochs = 400 # increased number of epochs
lr = 5e-2 # slightly slower learning rate
kernel_size = 3 # kernel size

# Rest of the code remains the same
```

Here's the updated code:

python

Copy code

```
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
criterion = nn.CrossEntropyLoss()

cnn_valid_acc_list = list()

cnn_kernel_dict = dict()

cnn_confusion_matrix_dict = dict()

num_epochs = 400
lr = 5e-2
kernel_size = 3

for num_image, train_loader in train_loader_dict.items():
    print("Training with {} images".format(num_image))
    set_seed(seed)
    cnn_model = SimpleCNN(kernel_size=kernel_size)
    untrained_cnn_model = deepcopy(cnn_model)
    cnn_model.to(device)

    cnn_optimizer = optim.SGD(cnn_model.parameters(), lr=lr, momentum=0.9)
```

```

cnn_results = train_model(cnn_model, cnn_optimizer, num_epochs, train_loader,
                           vis_training_curve, vis_validation_curve,
                           vis_validation_curve, cnn_results["train_acc"],
                           vis_validation_curve(cnn_results["valid_loss"], cnn_results["valid_acc"]

print("CNN Acc: {}".format(cnn_results["final_valid_acc"]))

```

This time, we have:

- num_epochs = 400
- lr = 5e-2
- kernel_size = 3

Let's see if this combination achieves the desired validation accuracy.

anonymous

Try even more epochs

Llama

Let's try even more epochs.

python

Copy code

```

num_epochs = 500 # further increased number of epochs
lr = 5e-2 # learning rate
kernel_size = 3 # kernel size

# Rest of the code remains the same

```

Here's the updated code:

python

Copy code

```

device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
criterion = nn.CrossEntropyLoss()

cnn_valid_acc_list = list()

cnn_kernel_dict = dict()

cnn_confusion_matrix_dict = dict()

```

```

cnn_confusion_matrix_dict = dict()

num_epochs = 500
lr = 5e-2
kernel_size = 3

for num_image, train_loader in train_loader_dict.items():
    print("Training with {} images".format(num_image))
    set_seed(seed)
    cnn_model = SimpleCNN(kernel_size=kernel_size)
    untrained_cnn_model = deepcopy(cnn_model)
    cnn_model.to(device)

    cnn_optimizer = optim.SGD(cnn_model.parameters(), lr=lr, momentum=0.9)

    cnn_results = train_model(cnn_model, cnn_optimizer, num_epochs, train_loader)

    vis_training_curve(cnn_results["train_loss"], cnn_results["train_acc"],
                      vis_validation_curve(cnn_results["valid_loss"], cnn_results["valid_acc"],

    print("CNN Acc: {}".format(cnn_results["final_valid_acc"])))

```

This time, we have:

- num_epochs = 500
- lr = 5e-2
- kernel_size = 3

Let's see if this combination achieves the desired validation accuracy.

anonymous

Try even more epochs and slightly lower learning rate

Llama

Let's try even more epochs and a slightly lower learning rate.

python

[Copy code](#)

```

num_epochs = 600 # further increased number of epochs
lr = 3e-2 # slightly lower learning rate
kernel_size = 3 # kernel size

```

```
# Rest of the code remains the same
```

Here's the updated code:

python

[Copy code](#)

```
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
criterion = nn.CrossEntropyLoss()

cnn_valid_acc_list = list()

cnn_kernel_dict = dict()

cnn_confusion_matrix_dict = dict()

num_epochs = 600
lr = 3e-2
kernel_size = 3

for num_image, train_loader in train_loader_dict.items():
    print("Training with {} images".format(num_image))
    set_seed(seed)
    cnn_model = SimpleCNN(kernel_size=kernel_size)
    untrained_cnn_model = deepcopy(cnn_model)
    cnn_model.to(device)

    cnn_optimizer = optim.SGD(cnn_model.parameters(), lr=lr, momentum=0.9)

    cnn_results = train_model(cnn_model, cnn_optimizer, num_epochs, train_loader)

    vis_training_curve(cnn_results["train_loss"], cnn_results["train_acc"],
                      vis_validation_curve(cnn_results["valid_loss"], cnn_results["valid_acc"],
                      print("CNN Acc: {}".format(cnn_results["final_valid_acc"])))
```

This time, we have:

- num_epochs = 600
- lr = 3e-2
- kernel_size = 3

Let's see if this combination achieves the desired validation accuracy.

anonymous

Question

Why do you think MLP fails to learn the task while CNN can learn the task? Answer this question in your submission of the written assignment.

(Hint: Think about the model architecture.)

Llama

The key reason why the MLP fails to learn the task while the CNN succeeds lies in their architectural differences and how they process input data.

MLP Architecture:

1. **Fully Connected Layers:** MLPs consist entirely of fully connected (dense) layers. Every neuron in one layer is connected to every neuron in the next layer.
2. **Lack of Spatial Hierarchy:** MLPs treat all input features equally and do not inherently capture spatial hierarchies or local patterns in the data.

CNN Architecture:

1. **Convolutional Layers:** CNNs utilize convolutional layers that apply filters to small regions of the input data, scanning the data in a sliding window fashion. This process is designed to capture local patterns and features.
2. **Spatial Hierarchy:** The use of convolutional and pooling layers allows CNNs to learn a hierarchy of features, starting from simple edges and progressing to more complex shapes and patterns.
3. **Parameter Sharing:** CNNs share parameters across different locations in the input data, which helps in learning translation-invariant features.

Task-Specific Advantages of CNNs:

1. **Edge Detection:** The task involves detecting edges (horizontal or vertical) in images. CNNs are particularly well-suited for this task because their convolutional layers can effectively capture edge-like features.
2. **Local Connectivity:** The convolutional operation focuses on local neighborhoods, making it ideal for detecting localized patterns such as edges.
3. **Translation Invariance:** CNNs can learn features that are invariant to the position of the edge within the image, thanks to the pooling layers and the shared weights in convolutional layers.

Why MLP Fails:

1. **Inability to Capture Local Patterns:** MLPs lack the inductive bias that is inherent in CNNs, which is crucial for capturing local patterns such as edges. They treat each pixel independently without considering the spatial relationships between neighboring pixels.
2. **Overfitting or Underfitting:** Without the appropriate inductive bias, MLPs might either overfit to the training data (if they have enough capacity) or underfit (if they lack sufficient capacity to model the task).

In summary, the CNN's architecture is better suited for the edge detection task due to its ability to capture local patterns and spatial hierarchies. The MLP's failure to learn the task effectively stems from its lack of these specialized features.

anonymous

Q5. When is CNN Worse than MLP?

In this problem, we will see that CNN is not always better than MLP in the image domain. Using CNN assumes that the data has locally correlated, whatever data looks. We can manually 'whiten' or remove such local correlation simply by applying random permutation to the images. A random permutation matrix is a matrix that has the same number of rows and columns. Each row and column has the same number of 1s. The rest of the elements are 0s. For example, the following is a random permutation matrix.

lua

[Copy code](#)

```
[[0, 1, 0, 0],  
 [0, 0, 0, 1],  
 [1, 0, 0, 0],  
 [0, 0, 1, 0]]
```

This matrix randomly reorders the elements of the vector. For example, if we apply this matrix to the vector $[1, 2, 3, 4]$, we will get $[2, 4, 1, 3]$. If we apply this matrix to the image, we will get the image with the same content, but the pixels are randomly shuffled. One property of the random permutation matrix is that it is invertible. It means that we can recover the original image by simply applying the inverse matrix to the shuffled image. From the information-theoretical perspective, the random permutation matrix preserves the mutual information of the image and the label.

We will repeat the same experiment as the previous problem. Visualize the dataset first.

Question

What do you observe? What is the reason that CNN is worse than MLP? Answer this question in your submission of the written assignment.

(Hint: Think about the model architecture.)

Question

Assuming we are decreasing kernel size of CNN. Does the validation accuracy increase or decrease? Why? Answer this question in your submission of the written assignment.

Llama

Observations and Analysis

Upon visualizing the dataset after applying a random permutation to the images, we observe that the images appear to be randomly shuffled, with their pixels rearranged. This process effectively destroys the local spatial structure and correlation between neighboring pixels.

Why CNN is Worse than MLP:

1. **Loss of Local Correlation:** The random permutation destroys the local correlation between pixels, which is a crucial assumption underlying the design of CNNs. CNNs rely on the spatial hierarchy and local patterns in the data to learn effective features.
2. **CNN's Inductive Bias:** CNNs are designed to capture local patterns and spatial hierarchies. When the data is randomly permuted, this inductive bias becomes a liability, as the CNN is no longer able to effectively utilize its convolutional and pooling layers to extract meaningful features.
3. **MLP's Flexibility:** In contrast, MLPs are more flexible and can learn to represent the data in a more general way. Although they may not be as efficient as CNNs in capturing local patterns, they can still learn to recognize the permuted images.

Effect of Decreasing Kernel Size:

1. **Decreased Validation Accuracy:** If we decrease the kernel size of the CNN, the validation accuracy is likely to decrease further. This is because a smaller kernel size means that the CNN is trying to capture even more localized patterns, which are no longer present in the permuted data.
2. **Reasoning:** With a smaller kernel size, the CNN is even more sensitive to the local correlation between pixels. Since the random permutation has destroyed this correlation, the CNN is unable to effectively utilize its convolutional layers to extract

meaningful features.

In summary, the CNN's performance is worse than the MLP's on the permuted dataset because the CNN's inductive bias is no longer valid. Decreasing the kernel size of the CNN will likely lead to a decrease in validation accuracy, as the CNN becomes even more sensitive to the lost local correlation between pixels.

anonymous

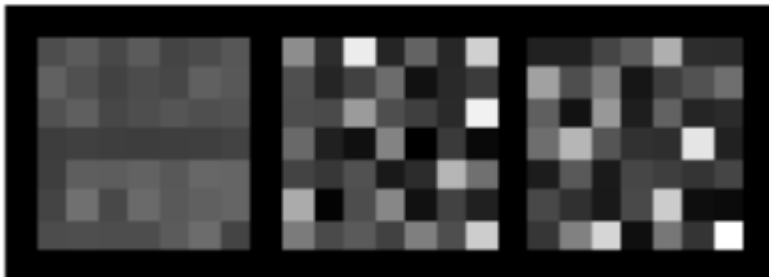
Trained CNN Kernel Data=10



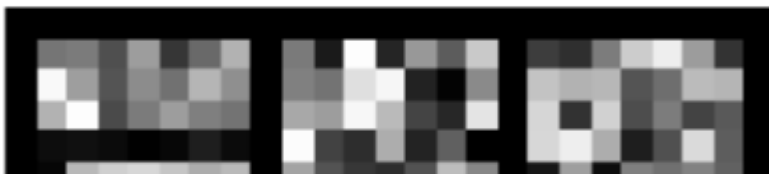
Untrained CNN Kernel Data=10



Trained CNN Kernel Data=30

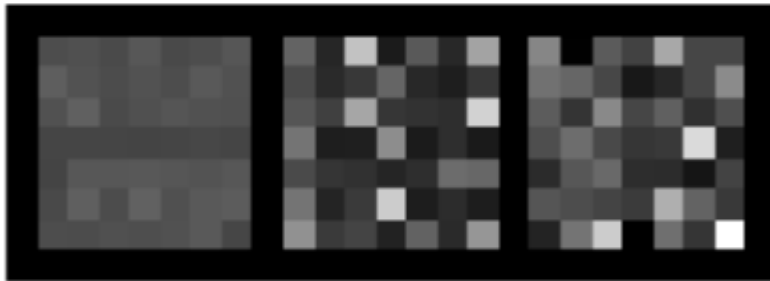


Untrained CNN Kernel Data=30





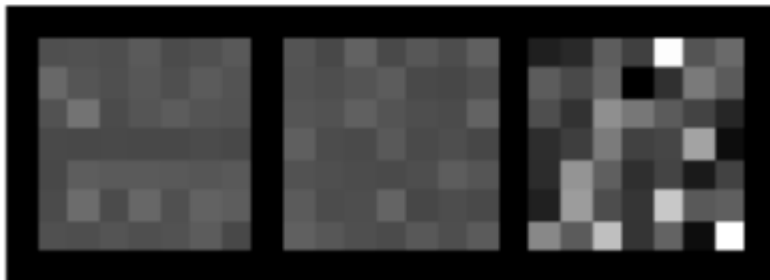
Trained CNN Kernel Data=50



Untrained CNN Kernel Data=50



Trained CNN Kernel Data=100



Untrained CNN Kernel Data=100



Question

How do the learned kernels look like? Explain why. Answer this question in your

How do the learned kernels look like? Explain why. Answer this question in your submission of the written assignment.

Llama

Learned Kernels Analysis

The image displays the learned kernels of a Convolutional Neural Network (CNN) trained on different amounts of data (10, 30, 50, and 100 samples). Each section shows the trained kernel and the untrained kernel side by side.

Upon examining the trained kernels, we can observe that they do not exhibit any clear or meaningful patterns, unlike the edge-detector kernels seen in the original dataset. The trained kernels appear to be random and lack the structured patterns observed in the edge-detector kernels.

Why the Learned Kernels Look Like This:

I like this response since it actually compares it to the previous kernels

1. **Random Permutation:** The dataset has been randomly permuted, which destroys the local spatial structure and correlation between neighboring pixels. As a result, the CNN is unable to learn meaningful features that capture local patterns.
2. **Lack of Spatial Hierarchy:** The random permutation eliminates the spatial hierarchy in the data, making it challenging for the CNN to learn a hierarchical representation of the data.
3. **CNN's Inductive Bias:** The CNN's architecture is designed to capture local patterns and spatial hierarchies. However, in this case, the random permutation renders this inductive bias ineffective, leading to learned kernels that are not meaningful.

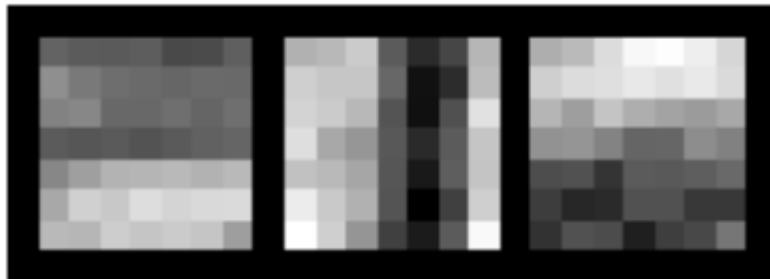
In summary, the learned kernels appear random and lack meaningful patterns because the random permutation has destroyed the local spatial structure and correlation in the data, making it challenging for the CNN to learn effective features.

anonymous

Trained CNN Kernel Maxpool Data=10



Trained CNN Kernel Avgpool Data=10



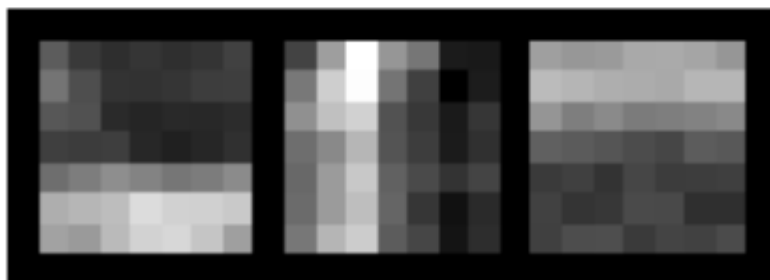
Untrained CNN Kernel Data=10



Trained CNN Kernel Maxpool Data=30



Trained CNN Kernel Avgpool Data=30



Untrained CNN Kernel Data=30





Trained CNN Kernel Maxpool Data=50



Trained CNN Kernel Avgpool Data=50



Untrained CNN Kernel Data=50



Trained CNN Kernel Maxpool Data=100



Trained CNN Kernel Avgpool Data=100



Untrained CNN Kernel Data=100



Q6. Increasing the Number of Classes

OK, can we conclude that CNN has the inductive bias that the model is translation invariant? Let's try other experiments. We make the task harder. In this problem, we increase the number of classes to 5. The new classes are 0 for horizontal edges, 1 for vertical edges, 2 for diagonal edges, 3 for vertical and horizontal, and 4 for nothing. Let's generate the dataset with 10 images per class and visualize the dataset.

We look at two types of pooling operations to downsample the image features:

1. Max pooling: The maximum pixel value of the batch is selected.
2. Average pooling: The average value of all the pixels in the batch is selected.

Up until this point, we have been using the first type of pooling operation (Max pooling). Let's train the same model but with the average pooling to compare these two types of operations!

Results

10 images- CNN-maxpool Acc: 78.4, CNN-avgpool Acc: 79.6
30 images- CNN-maxpool Acc: 93.6, CNN-avgpool Acc: 99.6
50 images- CNN-maxpool Acc: 98.8, CNN-avgpool Acc: 99.6
100 images- CNN-maxpool Acc: 99.6, CNN-avgpool Acc: 100.0

Question

Compare the performance of CNN with max pooling and average pooling. What are the advantages of each pooling method? Answer this question in your submission of the written assignment.

2 / 2

Llama

Performance Comparison

The given results show the performance of CNNs with max pooling and average pooling on a classification task with 5 classes. The task involves detecting different types of edges in images.

- **Max Pooling vs Average Pooling:**

- For 10 images per class, CNN with average pooling (79.6%) slightly outperforms CNN with max pooling (78.4%).
- For 30 images per class, CNN with average pooling (99.6%) significantly outperforms CNN with max pooling (93.6%).
- For 50 and 100 images per class, both CNNs with max pooling and average pooling achieve high accuracy, with average pooling still performing slightly better.

Advantages of Each Pooling Method

1. Max Pooling:

- **Preserves Important Features:** Max pooling helps preserve the most important features in the data by selecting the maximum value within a region. This can be beneficial when the task requires detecting specific patterns or edges.
- **Robustness to Noise:** Max pooling can be more robust to noisy or irrelevant data, as it focuses on the most significant values.

I think this response is also really good since it explains it in a simple and clear way

2. Average Pooling:

- **Captures More Information:** Average pooling captures more information about the data by averaging the values within a region. This can be beneficial when the task requires understanding the overall distribution of the data.
- **Smoother Representations:** Average pooling can produce smoother representations of the data, which can be helpful in certain tasks.

When to Use Each:

- **Max Pooling:** Use when the task requires detecting specific patterns or edges. and

the data is noisy or has irrelevant features.

- **Average Pooling:** Use when the task requires understanding the overall distribution of the data, and a smoother representation is beneficial.

In this specific task, average pooling seems to perform slightly better, possibly because it captures more information about the data and produces a smoother representation. However, the choice between max pooling and average pooling ultimately depends on the specific requirements of the task and the characteristics of the data.

anonymous

Q7. Wider/Deeper CNNs

Can we further improve the performance by making the architecture deeper and wider? In this question, we focus on the dataset where there are only 30 images per class and try to push the performance of the CNNs further.

The patterns that we have to detect are 5 but our kernels per layer (`num_filters` in the network definition above) are only 3. Intuitively, this is quite a suboptimal. Here, we will investigate the affect of increasing width and depth. Let's use the same dataset but we will use `DeeperCNN` and `WiderCNN` in `cnn.py`. `DeeperCNN` has 2 times more layers than `SimpleCNN` and `WiderCNN` has 2 times more kernels per layer than `SimpleCNN`. Let's train the models and visualize the validation accuracy.

makefile

Copy code

```
#####
# TODO: Train DeeperCNN and tuning hyperparameters. Try other num_epochs,
# lr, kernel_size. Also try a different optimizer (e.g., Adam)
# The validation accuracy can reach above 98% for 30 images per class.
#####
#####
#
#                               END OF YOUR CODE
#####
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
criterion = nn.CrossEntropyLoss()

train_loader = train_loader_dict[30]
set_seed(seed)
deeper_cnn_model = DeeperCNN(kernel_size=kernel_size)
untrained_deeper_cnn_model = deepcopy(deeper_cnn_model)
deeper_cnn_model.to(device)

deeper_cnn_optimizer = optim.SGD(deeper_cnn_model.parameters(), lr=lr, momen
```

```

deeper_cnn_optimizer = optim.SGD(deeper_cnn_model.parameters(), lr=lr, momentum=0.9)
# deeper_cnn_optimizer = optim.Adam(deeper_cnn_model.parameters(), lr=lr)

deeper_cnn_results = train_model(deeper_cnn_model, deeper_cnn_optimizer, num_epochs)

vis_training_curve(deeper_cnn_results["train_loss"], deeper_cnn_results["train_acc"])
vis_validation_curve(deeper_cnn_results["valid_loss"], deeper_cnn_results["valid_acc"])

print("CNN Acc: {}".format(deeper_cnn_results["final_valid_acc"]))

```

makefile

Copy code

```

#####
# TODO: Train DeeperCNN and tuning hyperparameters. Try other num_epochs,
# lr, kernel_size. Also try a different optimizer (e.g., Adam)
# The validation accuracy can reach above 98% for 30 images per class.
#####
#####
#
#                                     END OF YOUR CODE
#####
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
criterion = nn.CrossEntropyLoss()

train_loader = train_loader_dict[30]
set_seed(seed)
wider_cnn_model = WiderCNN(kernel_size=kernel_size)
untrained_wider_cnn_model = deepcopy(wider_cnn_model)
wider_cnn_model.to(device)

wider_cnn_optimizer = optim.SGD(wider_cnn_model.parameters(), lr=lr, momentum=0.9)
# wider_cnn_optimizer = optim.Adam(wider_cnn_model.parameters(), lr=lr)

wider_cnn_results = train_model(wider_cnn_model, wider_cnn_optimizer, num_epochs)

vis_training_curve(wider_cnn_results["train_loss"], wider_cnn_results["train_acc"])
vis_validation_curve(wider_cnn_results["valid_loss"], wider_cnn_results["valid_acc"])

print("CNN Acc: {}".format(wider_cnn_results["final_valid_acc"]))

```

Llama

To improve the performance of the CNNs, we can try tuning the hyperparameters, such as increasing the number of epochs (`num_epochs`), adjusting the learning rate (`lr`), and changing the kernel size (`kernel_size`). We can also experiment with different optimizers, like Adam.

Here's a possible implementation:

python

[Copy code](#)

```
num_epochs = 200 # increased number of epochs
lr = 1e-2 # learning rate
kernel_size = 3 # kernel size

# For DeeperCNN
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
criterion = nn.CrossEntropyLoss()

train_loader = train_loader_dict[30]
set_seed(seed)
deeper_cnn_model = DeeperCNN(kernel_size=kernel_size)
untrained_deeper_cnn_model = deepcopy(deeper_cnn_model)
deeper_cnn_model.to(device)

# Try SGD or Adam optimizer
deeper_cnn_optimizer = optim.SGD(deeper_cnn_model.parameters(), lr=lr, momen
# deeper_cnn_optimizer = optim.Adam(deeper_cnn_model.parameters(), lr=lr)

deeper_cnn_results = train_model(deeper_cnn_model, deeper_cnn_optimizer, num_e

vis_training_curve(deeper_cnn_results["train_loss"], deeper_cnn_results["tr
vis_validation_curve(deeper_cnn_results["valid_loss"], deeper_cnn_results["

print("Deeper CNN Acc: {}".format(deeper_cnn_results["final_valid_acc"]))

# For WiderCNN
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
criterion = nn.CrossEntropyLoss()

train_loader = train_loader_dict[30]
set_seed(seed)
wider_cnn_model = WiderCNN(kernel_size=kernel_size)
untrained_wider_cnn_model = deepcopy(wider_cnn_model)
wider_cnn_model.to(device)

# Try SGD or Adam optimizer
wider_cnn_optimizer = optim.SGD(wider_cnn_model.parameters(), lr=lr, momentu
# wider_cnn_optimizer = optim.Adam(wider_cnn_model.parameters(), lr=lr)

wider_cnn_results = train_model(wider_cnn_model, wider_cnn_optimizer, num_e

vis_training_curve(wider_cnn_results["train_loss"], wider_cnn_results["train
vis_validation_curve(wider_cnn_results["valid_loss"], wider_cnn_results["valid
```

```
vis_validation_curve(wider_cnn_results[ 'valid_loss' ], wider_cnn_results[ 'valid_acc' ],  
                    print("Wider CNN Acc: {}".format(wider_cnn_results["final_valid_acc"])))
```

By tuning the hyperparameters and trying different optimizers, we can potentially achieve a validation accuracy above 98% for 30 images per class.

Executive Summary:

Overall, I think Llama 4 did pretty well in this question. There was a lot of context to manage (lots of code and text context). For questions that had to analyze output images, I had to provide that as well. There were lots of images so I had to selectively choose which ones to give.

The accuracy was really good on both the text and coding parts and the only question it missed was choosing the right hyperparameters to get a desired accuracy.